

基于动态优化思想的离散系统的LQR最优控制推导

离散时间LQR问题的完整推导：从偏导到动态规划

看到复杂公式不要慌，思路很简单。求函数的极值，就是求偏导的问题

1. 问题设定

考虑离散时间线性系统：

$$x_{[k+1]} = Ax_{[k]} + Bu_{[k]}, \quad k = 1, 2, \dots, N-1$$

其中 $x_{[k]} \in \mathbb{R}^n$ 为状态向量， $u_{[k]} \in \mathbb{R}^m$ 为控制输入向量，初始状态 $x_{[1]}$ 给定。

有限时域代价函数（取 $N = 3$ 为例）：

$$J = x_{[3]}^T Q_N x_{[3]} + x_{[2]}^T Q x_{[2]} + u_{[2]}^T R u_{[2]} + x_{[1]}^T Q x_{[1]} + u_{[1]}^T R u_{[1]}$$

其中 $Q_N, Q \succeq 0$ 为状态权重矩阵（对称半正定）， $R \succ 0$ 为控制权重矩阵（对称正定）。

2. 逆向优化：先优化 $u_{[2]}$

2.1 分离与 $u_{[2]}$ 相关的项

将代价函数重写为：

$$J = \underbrace{x_{[3]}^T Q_N x_{[3]} + x_{[2]}^T Q x_{[2]} + u_{[2]}^T R u_{[2]}}_{J_2} + x_{[1]}^T Q x_{[1]} + u_{[1]}^T R u_{[1]}$$

其中 J_2 包含所有与 $u_{[2]}$ 相关的项（注意 $x_{[3]}$ 依赖于 $u_{[2]}$ ）。

将状态方程 $x_{[3]} = Ax_{[2]} + Bu_{[2]}$ 代入 J_2 ：

$$\begin{aligned} J_2 &= (Ax_{[2]} + Bu_{[2]})^T Q_N (Ax_{[2]} + Bu_{[2]}) + x_{[2]}^T Q x_{[2]} + u_{[2]}^T R u_{[2]} \\ &= x_{[2]}^T A^T Q_N A x_{[2]} + x_{[2]}^T A^T Q_N B u_{[2]} + u_{[2]}^T B^T Q_N A x_{[2]} + u_{[2]}^T B^T Q_N B u_{[2]} + x_{[2]}^T Q x_{[2]} + u_{[2]}^T R u_{[2]} \\ &= x_{[2]}^T (A^T Q_N A + Q) x_{[2]} + 2x_{[2]}^T A^T Q_N B u_{[2]} + u_{[2]}^T (B^T Q_N B + R) u_{[2]} \end{aligned}$$

这里利用了 Q_N 的对称性，且 $x_{[2]}^T A^T Q_N B u_{[2]}$ 是标量，等于其转置 $u_{[2]}^T B^T Q_N A x_{[2]}$ ，故合并为 $2x_{[2]}^T A^T Q_N B u_{[2]}$ 。

2.2 对 $u_{[2]}$ 求偏导

$$\frac{\partial J_2}{\partial u_{[2]}} = 2(B^T Q_N B + R)u_{[2]} + 2B^T Q_N A x_{[2]}$$

令偏导为零：

$$(B^T Q_N B + R)u_{[2]} + B^T Q_N A x_{[2]} = 0$$

解得最优控制：

$$u_{[2]}^* = -(B^T Q_N B + R)^{-1} B^T Q_N A x_{[2]}$$

定义反馈增益：

$$K_2 = (B^T Q_N B + R)^{-1} B^T Q_N A$$

则 $u_{[2]}^* = -K_2 x_{[2]}$ 。

2.3 计算最小剩余代价（值函数）

将 $u_{[2]}^*$ 代回 J_2 ：

$$\begin{aligned} J_2^* &= \min_{u_{[2]}} J_2 \\ &= x_{[2]}^T (A^T Q_N A + Q) x_{[2]} + 2x_{[2]}^T A^T Q_N B (-K_2 x_{[2]}) + (-K_2 x_{[2]})^T (B^T Q_N B + R) (-K_2 x_{[2]}) \\ &= x_{[2]}^T [A^T Q_N A + Q - 2A^T Q_N B K_2 + K_2^T (B^T Q_N B + R) K_2] x_{[2]} \end{aligned}$$

代入 $K_2 = (B^T Q_N B + R)^{-1} B^T Q_N A$ ，计算：

$$\begin{aligned} K_2^T (B^T Q_N B + R) K_2 &= (B^T Q_N A)^T (B^T Q_N B + R)^{-1} (B^T Q_N B + R) (B^T Q_N B + R)^{-1} B^T Q_N A \\ &= (B^T Q_N A)^T (B^T Q_N B + R)^{-1} B^T Q_N A \quad (\text{因 } B^T Q_N B + R \text{ 对称}) \\ &= A^T Q_N B (B^T Q_N B + R)^{-1} B^T Q_N A \end{aligned}$$

又因 $A^T Q_N B K_2 = A^T Q_N B (B^T Q_N B + R)^{-1} B^T Q_N A$ ，故：

$$-2A^T Q_N B K_2 + K_2^T (B^T Q_N B + R) K_2 = -A^T Q_N B (B^T Q_N B + R)^{-1} B^T Q_N A$$

因此：

$$J_2^* = x_{[2]}^T [A^T Q_N A + Q - A^T Q_N B (B^T Q_N B + R)^{-1} B^T Q_N A] x_{[2]}$$

定义：

$$P_2 = A^T Q_N A + Q - A^T Q_N B (B^T Q_N B + R)^{-1} B^T Q_N A$$

则：

$$J_2^* = x_{[2]}^T P_2 x_{[2]}$$

矩阵 P_2 包含了从时刻 2 到终端 3 的最小代价信息，它将未来的代价压缩成一个二次型。

3. 再优化 $u_{[1]}$

3.1 利用已得的 P_2 重写总代价

将 J_2^* 代回总代价 J ：

$$J = x_{[2]}^T P_2 x_{[2]} + x_{[1]}^T Q x_{[1]} + u_{[1]}^T R u_{[1]}$$

注意 $x_{[2]} = Ax_{[1]} + Bu_{[1]}$ ，代入：

$$\begin{aligned} J &= (Ax_{[1]} + Bu_{[1]})^T P_2 (Ax_{[1]} + Bu_{[1]}) + x_{[1]}^T Q x_{[1]} + u_{[1]}^T R u_{[1]} \\ &= x_{[1]}^T A^T P_2 A x_{[1]} + x_{[1]}^T A^T P_2 B u_{[1]} + u_{[1]}^T B^T P_2 A x_{[1]} + u_{[1]}^T B^T P_2 B u_{[1]} \\ &\quad + x_{[1]}^T Q x_{[1]} + u_{[1]}^T R u_{[1]} \\ &= x_{[1]}^T (A^T P_2 A + Q) x_{[1]} + 2x_{[1]}^T A^T P_2 B u_{[1]} + u_{[1]}^T (B^T P_2 B + R) u_{[1]} \end{aligned}$$

关键观察：此时形式与优化 $u_{[2]}$ 时的 J_2 完全类似，只是将 Q_N 替换为 P_2 ，将 $x_{[2]}$ 替换为 $x_{[1]}$ 。这正是动态规划的精髓：**后续的代价被压缩进矩阵 P_2 中**，使得当前步的优化具有统一的形式。

3.2 对 $u_{[1]}$ 求偏导

$$\frac{\partial J}{\partial u_{[1]}} = 2(B^T P_2 B + R)u_{[1]} + 2B^T P_2 A x_{[1]}$$

令偏导为零：

$$(B^T P_2 B + R)u_{[1]} + B^T P_2 A x_{[1]} = 0$$

解得：

$$u_{[1]}^* = -(B^T P_2 B + R)^{-1} B^T P_2 A x_{[1]}$$

定义： $K_1 = (B^T P_2 B + R)^{-1} B^T P_2 A$

则 $u_{[1]}^* = -K_1 x_{[1]}$ 。

3.3 计算全局最小代价

将 $u_{[1]}^*$ 代回 J ：

$$\begin{aligned} J^* &= x_{[1]}^T (A^T P_2 A + Q) x_{[1]} + 2x_{[1]}^T A^T P_2 B (-K_1 x_{[1]}) + (-K_1 x_{[1]})^T (B^T P_2 B + R) (-K_1 x_{[1]}) \\ &= x_{[1]}^T [A^T P_2 A + Q - 2A^T P_2 B K_1 + K_1^T (B^T P_2 B + R) K_1] x_{[1]} \end{aligned}$$

代入 $K_1 = (B^T P_2 B + R)^{-1} B^T P_2 A$ ，类似可得：

$$K_1^T (B^T P_2 B + R) K_1 = A^T P_2 B (B^T P_2 B + R)^{-1} B^T P_2 A$$

且 $A^T P_2 B K_1 = A^T P_2 B (B^T P_2 B + R)^{-1} B^T P_2 A$ ，故：

$$-2A^T P_2 B K_1 + K_1^T (B^T P_2 B + R) K_1 = -A^T P_2 B (B^T P_2 B + R)^{-1} B^T P_2 A$$

因此：

$$J^* = x_{[1]}^T [A^T P_2 A + Q - A^T P_2 B (B^T P_2 B + R)^{-1} B^T P_2 A] x_{[1]}$$

定义：
$$P_1 = A^T P_2 A + Q - A^T P_2 B (B^T P_2 B + R)^{-1} B^T P_2 A$$

则：
$$J^* = x_{[1]}^T P_1 x_{[1]}$$

4. 动态规划与Riccati方程归纳

从上述推导可归纳出一般递推公式。定义**值函数** $V_k(x_{[k]})$ 为从时刻 k 到终端的最小剩余代价：

$$V_k(x_{[k]}) = \min_{u_{[k]}, \dots, u_{[N-1]}} \left[x_{[N]}^T Q_N x_{[N]} + \sum_{i=k}^{N-1} (x_{[i]}^T Q x_{[i]} + u_{[i]}^T R u_{[i]}) \right]$$

假设其具有二次型形式：

$$V_k(x_{[k]}) = x_{[k]}^T P_k x_{[k]}, \quad P_k = P_k^T \succeq 0$$

终端条件： $V_N(x_{[N]}) = x_{[N]}^T Q_N x_{[N]}$ ，即 $P_N = Q_N$ 。

由Bellman最优性方程：

$$V_k(x_{[k]}) = \min_{u_{[k]}} \left[x_{[k]}^T Q x_{[k]} + u_{[k]}^T R u_{[k]} + V_{k+1}(x_{[k+1]}) \right]$$

代入 $x_{[k+1]} = Ax_{[k]} + Bu_{[k]}$ 和 $V_{k+1}(x) = x^T P_{k+1} x$ ：

$$V_k(x_{[k]}) = \min_{u_{[k]}} \left[x_{[k]}^T Q x_{[k]} + u_{[k]}^T R u_{[k]} + (Ax_{[k]} + Bu_{[k]})^T P_{k+1} (Ax_{[k]} + Bu_{[k]}) \right]$$

展开并合并同类项：

$$V_k(x_{[k]}) = \min_{u_{[k]}} \left[x_{[k]}^T (Q + A^T P_{k+1} A) x_{[k]} + 2x_{[k]}^T A^T P_{k+1} B u_{[k]} + u_{[k]}^T (R + B^T P_{k+1} B) u_{[k]} \right]$$

对 $u_{[k]}$ 求偏导并令为零：

$$\frac{\partial}{\partial u_{[k]}} [\dots] = 2(R + B^T P_{k+1} B) u_{[k]} + 2B^T P_{k+1} A x_{[k]} = 0$$

解得最优控制律：

$$u_{[k]}^* = -(R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A x_{[k]} = -K_k x_{[k]}$$

其中：

$$K_k = (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A$$

将 $u_{[k]}^*$ 代回，得到离散时间Riccati方程：

$$P_k = Q + A^T P_{k+1} A - A^T P_{k+1} B (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A \quad (\text{一些书上 } P_{k+1} \text{ 和 } P_k \text{ 的位置相反, 是因为定义的不同})$$

也可写作：

$$P_k = Q + A^T P_{k+1} (A - BK_k) = Q + (A - BK_k)^T P_{k+1} (A - BK_k) + K_k^T R K_k$$

5. 算法流程

1. 初始化：设 $P_N = Q_N$

2. 逆向递推：对 $k = N-1, N-2, \dots, 1$ ：

$$K_k = (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A$$
$$P_k = Q + A^T P_{k+1} A - A^T P_{k+1} B K_k$$

3. 正向控制：对 $k = 1, 2, \dots, N - 1$ ：

$$u_{[k]}^* = -K_k x_{[k]}$$

6. 核心思想总结

步骤	优化变量	代价函数中的未来项	结果矩阵	最优控制律形式
第一步	$u_{[2]}$	$x_{[3]}^T Q_N x_{[3]}$	$P_2 = A^T Q_N A + Q - A^T Q_N B (\dots)^{-1} B^T Q_N A$	$u_{[2]}^* = -K_2 x_{[2]}$
第二步	$u_{[1]}$	$x_{[2]}^T P_2 x_{[2]}$	$P_1 = A^T P_2 A + Q - A^T P_2 B (\dots)^{-1} B^T P_2 A$	$u_{[1]}^* = -K_1 x_{[1]}$

精妙之处：

1. 逆向递推：从最后一刻向前优化，每一步的优化都将后续的所有代价信息压缩进一个矩阵 P_k 中。

2. 统一结构：每一步的优化问题都具有完全相同的形式：

$$\frac{\partial J}{\partial u_{[k]}} = 2(R + B^T P_{\text{future}} B) u_{[k]} + 2B^T P_{\text{future}} A x_{[k]} = 0$$

其中 P_{future} 是下一步的代价矩阵（ P_{k+1} 或 Q_N ）。

3. 动态规划本质：值函数 $V_k(x) = x^T P_k x$ 满足 Bellman 方程，其二次型结构在 LQR 问题中得以保持，从而导出了 Riccati 方程的递推关系。

这种推导方法清晰地展示了如何从直接对控制序列求偏导（“时间空间化”视角）自然过渡到动态规划的逆向递推，最终得到经典的 Riccati 方程。

1. 三层转化框架

整个过程可以总结为三个层次的转化，形成一个完整的理论体系：

第一层：无限维 → 有限维（从泛函到函数）

- 连续时间问题**：在无限维函数空间 $u(t) \in \mathcal{U}$ 中寻找最优函数，是**变分问题**。
- 离散化后**：在有限维向量空间 $\mathbf{U} = [u_1, u_2, \dots, u_N]^T \in \mathbb{R}^{mN}$ 中寻找最优点，是**多元函数极值问题**。
- 关键洞见**：时间变量 t 从函数的**自变量**变成了向量的**分量索引**，即“时间空间化”。

第二层：高维耦合 → 低维递推（从整体优化到动态规划）

- 直接方法：对 $J(\mathbf{U})$ 同时求所有偏导数 $\frac{\partial J}{\partial u_k} = 0$ ，得到 mN 个耦合方程，计算复杂。
- 动态规划：利用问题的时间结构，逆向分解：

$$\underbrace{\min_{u_1, \dots, u_N} J}_{\text{高维问题}} \Rightarrow \underbrace{\min_{u_N} \left\{ \dots \min_{u_2} \left\{ \min_{u_1} J \right\} \right\}}_{\text{嵌套的低维问题}}$$

- 数学本质：Bellman最优性原理保证了这种分解不损失最优性。

第三层：开环序列 → 反馈策略（从离线规划到在线控制）

- 开环解：直接优化得到固定的序列 $\{u_1^*, u_2^*, \dots\}$ 。
- 反馈解：动态规划得到 策略 $u_k^* = -K_k x_k$ 。
- 根本优势：反馈解具有鲁棒性，能应对干扰和模型误差。

2. “局部最优等于全局最优” 的精确表述

更精确地表述：

动态规划的最优性原理：一个多步决策问题的最优策略具有这样的性质——无论初始状态和初始决策如何，剩余的决策必须构成以第一个决策产生的状态为初始状态的最优策略。

在离散LQR中，这表现为：

1. 逆向递推的合理性：从最后一步开始，假设我们已经知道从 $k + 1$ 步开始的最优代价函数 $V_{k+1}(x) = x^T P_{k+1} x$ 。

2. 单步优化的全局性：在第 k 步求解：

$$V_k(x_k) = \min_{u_k} [x_k^T Q x_k + u_k^T R u_k + V_{k+1}(x_{k+1})]$$

这个单步优化确实是局部问题，但它的“局部最优解” u_k^* 是全局最优解的一部分，因为：

- 它考虑了所有未来代价（压缩在 V_{k+1} 中）。
- 逆向递推保证了无后效性（Markov性）。

3. 严格的表述：在满足Markov性和可分性结构的优化问题中，通过动态规划得到的每一步局部最优决策，组合起来构成全局最优策略。这不是简单的“局部最优等于全局最优”，而是系统的结构性分解保证了局部最优解的兼容性与全局最优性。

3. 离散LQR的特殊性与一般性

特殊性（使得问题可解）：

1. 线性系统： $x_{k+1} = Ax_k + Bu_k$

2. 二次代价： $J = \sum (x^T Q x + u^T R u)$

3. 导致值函数保持二次型： $V_k(x) = x^T P_k x$

一般性（方法论意义）：

1. 动态规划思想适用于更广泛的序列决策问题。
2. 时间空间化视角是连接连续与离散、控制与优化的桥梁。
3. 从开环到反馈的设计哲学是控制理论的核心。

4. 完善后的整体表述

离散线性二次型调节器（LQR）问题的求解，展现了一个优美的三层转化框架：

1. **维度约简**：通过时间离散化，将连续时间泛函极值问题转化为有限维向量空间中的多元函数优化问题。这实现了从“寻找最优函数曲线”到“寻找最优向量点”的思维转换，即“时间空间化”。
2. **结构分解**：利用动态规划的最优性原理，将高维耦合优化问题按时间维度逆向分解为一系列低维单步优化子问题。每个子问题只优化当前时刻的控制输入，而未来所有步的代价信息被压缩进一个二次型值函数中。这种分解不损失最优性，因为Bellman原理保证了局部最优决策的可组合性。
3. **策略升华**：求解过程自然导出了状态反馈形式的最优控制律，将开环的轨迹规划提升为闭环的反馈策略，赋予了控制系统内在的鲁棒性。
4. 最终，这一系列转化在LQR中凝结为Riccati差分方程，它既是数学上的简洁表达，也是“分而治之”优化思想的完美体现。

5. 与您原表述的对照

您的原表述：

“先把系统离散化，把一个泛函问题转换成一个求解普通函数极值的问题；在普通函数极值求解问题中，又把一个高维的优化问题转换成很多个子空间进行分别的求解，每一个局部最优等于全局最优。”

完善后的表述：

“通过离散化实现从泛函到函数的维度约简，通过动态规划实现从高维耦合到低维递推的结构分解，最终利用问题的时间结构保证局部最优解可组合为全局最优策略，并自然导出反馈形式的控制律。”

这个思考框架不仅适用于LQR，也适用于强化学习、动态优化等领域，是序列决策问题的通用方法论。

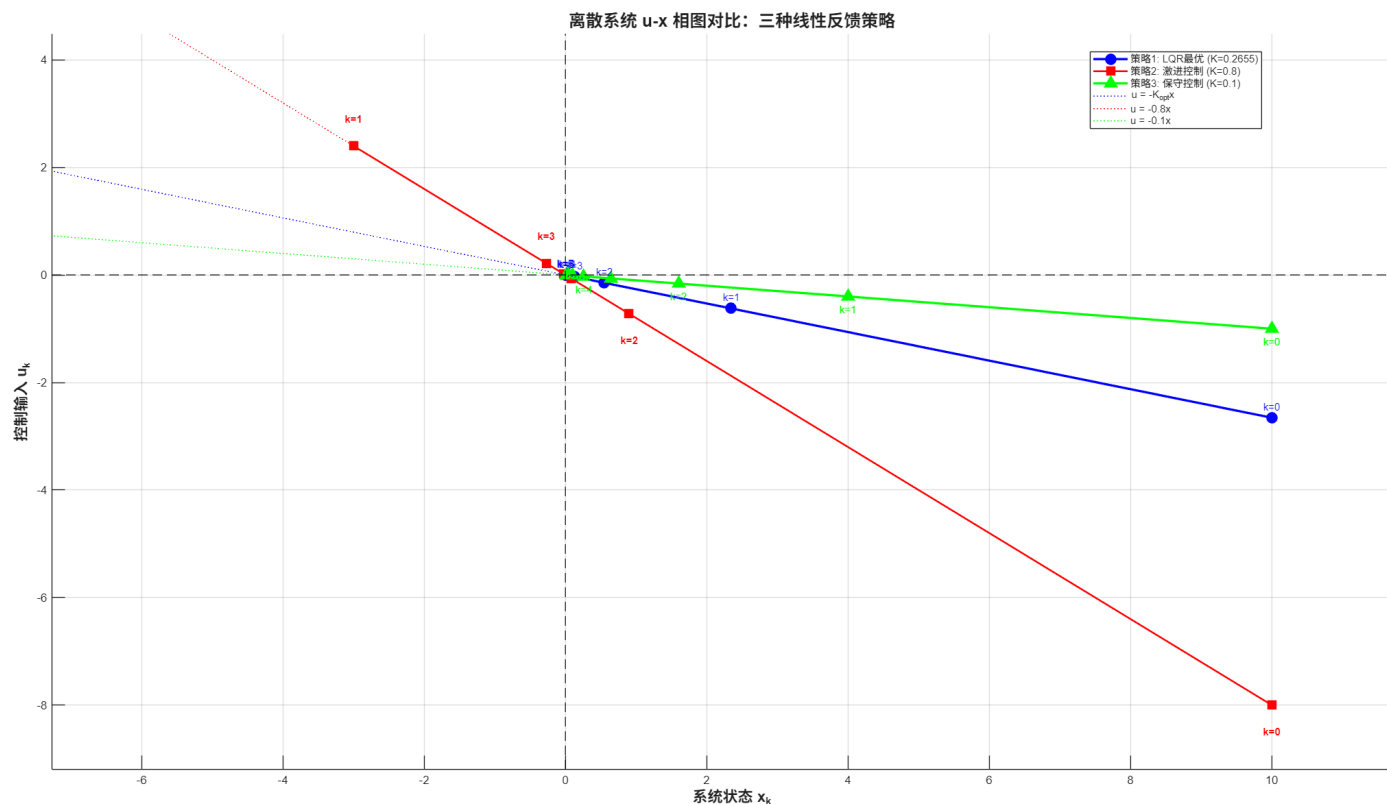
泛函的意义的可视化代码

系统是 $x_{k+1} = 0.5x_k + u_k$

1. 输入一个最优序列 $u_1, u_2, \dots, u_k$ 和另外两个保守和激进的序列，得到 $u-x$ 相图……

注意，相图上面绘制的都是单步代价函数，不是累计代价函数。合围的面积才是累计代价函数。

所以对 u 轴切片，得到的蓝色点不一定在最低点，因为不是累计代价函数画的。



代码块

```
1 % 第二步 (更新) : 对比三种控制策略的  $u-x$  相图
2 clear; clc; close all;
3
4 % 系统定义
5 A = 0.5;
6 B = 1;
7 x0 = 10;
8 N = 8; % 仿真步数
9
10 % 1. 策略1: LQR 最优控制
11 Q = 1; R = 1;
12 [K_opt, ~, ~] = dlqr(A, B, Q, R); % 最优反馈增益  $K_{opt} \approx 0.2655$ 
13 x_opt = zeros(1, N+1);
14 u_opt = zeros(1, N);
15 x_opt(1) = x0;
16 for k = 1:N
17     u_opt(k) = -K_opt * x_opt(k);
18     x_opt(k+1) = A * x_opt(k) + B * u_opt(k);
19 end
20
```



```

21 % 2. 策略2: 新激进控制 (增益  $K=0.8$ )
22 K_aggressive = 0.8;
23 x_agg = zeros(1, N+1);
24 u_agg = zeros(1, N);
25 x_agg(1) = x0;
26 for k = 1:N
27     u_agg(k) = -K_aggressive * x_agg(k);
28     x_agg(k+1) = A * x_agg(k) + B * u_agg(k);
29 end
30
31 % 3. 策略3: 保守控制 (增益  $K=0.1$ )
32 K_conservative = 0.1;
33 x_con = zeros(1, N+1);
34 u_con = zeros(1, N);
35 x_con(1) = x0;
36 for k = 1:N
37     u_con(k) = -K_conservative * x_con(k);
38     x_con(k+1) = A * x_con(k) + B * u_con(k);
39 end
40
41 %% 绘制 u-x 相图对比
42 figure('Color', 'w', 'Position', [200, 200, 750, 600]);
43 hold on; grid on;
44
45 % 绘制三条轨迹
46 p1 = plot(x_opt(1:N), u_opt, '-bo', 'LineWidth', 2, 'MarkerFaceColor', 'b',
47 'MarkerSize', 8, 'DisplayName', '策略1: LQR最优 ( $K=0.2655$ )');
48 p2 = plot(x_agg(1:N), u_agg, '-rs', 'LineWidth', 1.5, 'MarkerFaceColor', 'r',
49 'MarkerSize', 9, 'DisplayName', '策略2: 激进控制 ( $K=0.8$ )');
50 p3 = plot(x_con(1:N), u_con, '-g^', 'LineWidth', 2, 'MarkerFaceColor', 'g',
51 'MarkerSize', 8, 'DisplayName', '策略3: 保守控制 ( $K=0.1$ )');
52
53 % 为策略1 (LQR最优) 的每个点标注时刻k
54 for k = 1:N
55     text(x_opt(k), u_opt(k)+0.2, sprintf('k=%d', k-1), ...
56         'FontSize', 9, 'HorizontalAlignment', 'center', 'Color', 'b');
57 end
58
59 % 为策略2 (激进) 的前几个点标注时刻, 注意其状态会变号
60 for k = 1:min(4, N)
61     offset = 0.5 * sign(u_agg(k)); % 根据输入方向调整文本偏移
62     text(x_agg(k), u_agg(k)+offset, sprintf('k=%d', k-1), ...
63         'FontSize', 9, 'HorizontalAlignment', 'center', 'Color', 'r',
64         'FontWeight', 'bold');
65 end
66
67 % 为策略3 (保守) 的前几个点标注时刻
68 for k = [1,2,3,5]
69     text(x_con(k), u_con(k)-0.25, sprintf('k=%d', k-1), ...

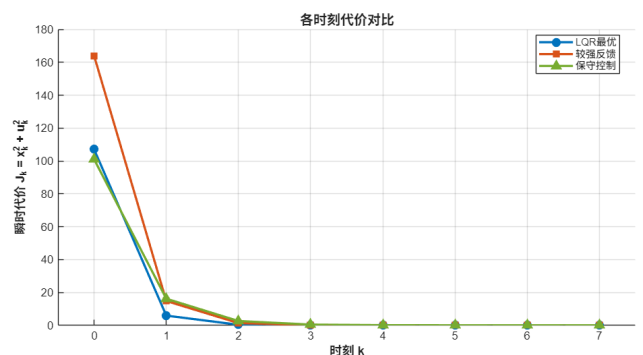
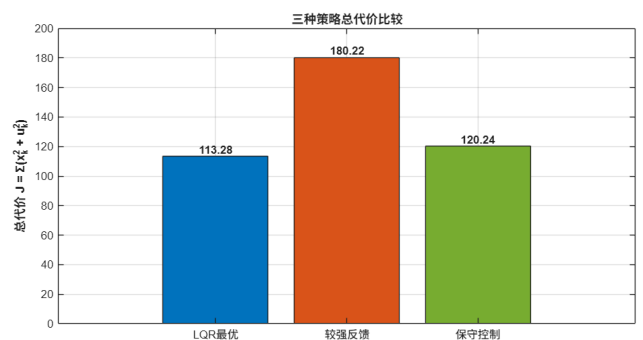
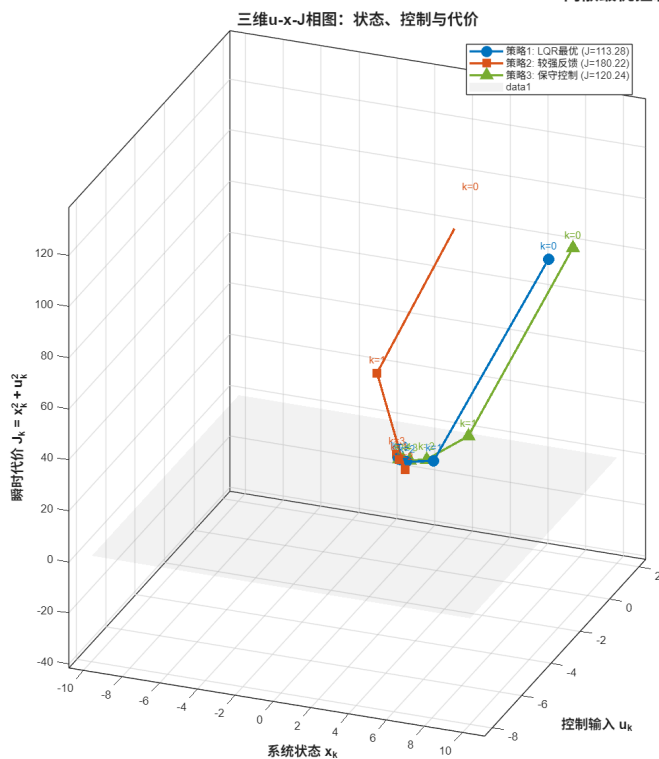
```

```

64         'FontSize', 9, 'HorizontalAlignment', 'center', 'Color', 'g');
65     end
66
67     % 绘制原点辅助线和"最优反馈线"
68     xline(0, 'k--', 'LineWidth', 1, 'HandleVisibility', 'off');
69     yline(0, 'k--', 'LineWidth', 1, 'HandleVisibility', 'off');
70     % 绘制三条控制律对应的直线
71     x_range = -10:0.1:10;
72     plot(x_range, -K_opt*x_range, 'b:', 'LineWidth', 0.8, 'DisplayName', 'u = -K_{opt}x');
73     plot(x_range, -K_aggressive*x_range, 'r:', 'LineWidth', 0.8, 'DisplayName', 'u = -0.8x');
74     plot(x_range, -K_conservative*x_range, 'g:', 'LineWidth', 0.8, 'DisplayName', 'u = -0.1x');
75
76     % 图形标注
77     xlabel('系统状态 x_k', 'FontSize', 13, 'FontWeight', 'bold');
78     ylabel('控制输入 u_k', 'FontSize', 13, 'FontWeight', 'bold');
79     title('离散系统 u-x 相图对比：三种线性反馈策略', 'FontSize', 15, 'FontWeight', 'bold');
80     legend('Location', 'northwest');
81     axis([-11, 11, -9, 2]); % 调整坐标轴以容纳振荡轨迹
82     hold off;

```

离散最优控制：代价函数分析与比较



代码块

1 %% 第三步：三维u-x-J相图 - 展示状态、控制与代价函数的关系

```

2  clear; clc; close all;
3
4  % 系统参数
5  A = 0.5;
6  B = 1;
7  x0 = 10;
8  N = 8; % 仿真步数
9  Q = 1; % 状态代价权重
10 R = 1; % 控制代价权重
11
12 % 计算LQR最优增益
13 [K_opt, ~, ~] = dlqr(A, B, Q, R); % 最优反馈增益  $K_{opt} \approx 0.2655$ 
14
15 % 1. 策略1: LQR最优控制
16 x_opt = zeros(1, N+1);
17 u_opt = zeros(1, N);
18 J_opt = zeros(1, N); % 存储每个时刻的代价函数值
19 x_opt(1) = x0;
20
21 for k = 1:N
22     u_opt(k) = -K_opt * x_opt(k);
23     J_opt(k) = x_opt(k)^2 + u_opt(k)^2; % 当前时刻的代价
24     x_opt(k+1) = A * x_opt(k) + B * u_opt(k);
25 end
26
27 % 2. 策略2: 较强反馈控制 ( $u = -0.8x$ )
28 K_strong = 0.8;
29 x_strong = zeros(1, N+1);
30 u_strong = zeros(1, N);
31 J_strong = zeros(1, N);
32 x_strong(1) = x0;
33
34 for k = 1:N
35     u_strong(k) = -K_strong * x_strong(k);
36     J_strong(k) = x_strong(k)^2 + u_strong(k)^2;
37     x_strong(k+1) = A * x_strong(k) + B * u_strong(k);
38 end
39
40 % 3. 策略3: 保守控制 ( $u = -0.1x$ )
41 K_con = 0.1;
42 x_con = zeros(1, N+1);
43 u_con = zeros(1, N);
44 J_con = zeros(1, N);
45 x_con(1) = x0;
46
47 for k = 1:N
48     u_con(k) = -K_con * x_con(k);

```

```

49     J_con(k) = x_con(k)^2 + u_con(k)^2;
50     x_con(k+1) = A * x_con(k) + B * u_con(k);
51 end
52
53 %% 计算总代价
54 total_cost_opt = sum(J_opt);
55 total_cost_strong = sum(J_strong);
56 total_cost_con = sum(J_con);
57
58 fprintf('=== 三种控制策略的总代价比较 (N=%d) ===\n', N);
59 fprintf('策略1 (LQR最优, K=%.4f): 总代价 J = %.4f\n', K_opt, total_cost_opt);
60 fprintf('策略2 (较强反馈, K=0.8): 总代价 J = %.4f\n', total_cost_strong);
61 fprintf('策略3 (保守控制, K=0.1): 总代价 J = %.4f\n', total_cost_con);
62 fprintf('\n代价排名:\n');
63 fprintf('1. 最优策略: %.4f\n', min([total_cost_opt, total_cost_strong,
total_cost_con]));
64 fprintf('2. 中等: %.4f\n', median([total_cost_opt, total_cost_strong,
total_cost_con]));
65 fprintf('3. 最差: %.4f\n', max([total_cost_opt, total_cost_strong,
total_cost_con]));
66
67 %% 绘制三维u-x-J相图
68 figure('Color', 'w', 'Position', [100, 100, 900, 700]);
69
70 % 创建三维坐标系
71 ax1 = subplot(2,2,[1,3]);
72 hold on; grid on; box on;
73 view(45, 30); % 设置视角
74
75 % 绘制三条轨迹
76 % 策略1: LQR最优 (蓝色)
77 plot3(x_opt(1:N), u_opt, J_opt, '-o', 'Color', [0, 0.4470, 0.7410], ...
78       'LineWidth', 2, 'MarkerFaceColor', [0, 0.4470, 0.7410], 'MarkerSize', 8,
79       ...
80       'DisplayName', sprintf('策略1: LQR最优 (J=%.2f)', total_cost_opt));
81
82 % 策略2: 较强反馈 (红色)
83 plot3(x_strong(1:N), u_strong, J_strong, '-s', 'Color', [0.8500, 0.3250,
0.0980], ...
84       'LineWidth', 2, 'MarkerFaceColor', [0.8500, 0.3250, 0.0980], 'MarkerSize',
85       8, ...
86       'DisplayName', sprintf('策略2: 较强反馈 (J=%.2f)', total_cost_strong));
87
88 % 策略3: 保守控制 (绿色)
89 plot3(x_con(1:N), u_con, J_con, '-^', 'Color', [0.4660, 0.6740, 0.1880], ...
90       'LineWidth', 2, 'MarkerFaceColor', [0.4660, 0.6740, 0.1880], 'MarkerSize',
91       8, ...

```

```

89     'DisplayName', sprintf('策略3: 保守控制 (J=%.2f)', total_cost_con));
90
91 % 为每个点标注时刻k
92 for k = 1:min(5, N) % 标注前5个点
93     text(x_opt(k), u_opt(k), J_opt(k)+5, sprintf('k=%d', k-1), ...
94         'FontSize', 9, 'HorizontalAlignment', 'center', 'Color', [0, 0.4470,
95         0.7410]));
96     text(x_strong(k), u_strong(k), J_strong(k)+5, sprintf('k=%d', k-1), ...
97         'FontSize', 9, 'HorizontalAlignment', 'center', 'Color', [0.8500,
98         0.3250, 0.0980]));
99     text(x_con(k), u_con(k), J_con(k)+5, sprintf('k=%d', k-1), ...
100         'FontSize', 9, 'HorizontalAlignment', 'center', 'Color', [0.4660,
101         0.6740, 0.1880]));
102 end
103
104 % 添加坐标轴标签
105 xlabel('系统状态 x_k', 'FontSize', 12, 'FontWeight', 'bold');
106 ylabel('控制输入 u_k', 'FontSize', 12, 'FontWeight', 'bold');
107 zlabel('瞬时代价 J_k = x_k^2 + u_k^2', 'FontSize', 12, 'FontWeight', 'bold');
108 title('三维u-x-J相图: 状态、控制与代价', 'FontSize', 14, 'FontWeight', 'bold');
109 legend('Location', 'best');
110 grid on;
111
112 % 设置坐标轴范围
113 xlim([-11, 11]);
114 ylim([-9, 2]);
115 zlim([0, max([J_opt, J_strong, J_con])*1.1]);
116
117 % 绘制x=0, u=0平面
118 [x_plane, u_plane] = meshgrid(-10:5:10, -8:2:2);
119 J_plane = zeros(size(x_plane));
120 surf(x_plane, u_plane, J_plane, 'FaceAlpha', 0.1, 'EdgeColor', 'none',
121     'FaceColor', [0.5, 0.5, 0.5]);
122
123 %% 绘制总代价比较柱状图
124 ax2 = subplot(2,2,2);
125 strategies = {'LQR最优', '较强反馈', '保守控制'};
126 total_costs = [total_cost_opt, total_cost_strong, total_cost_con];
127 colors = {[0, 0.4470, 0.7410], [0.8500, 0.3250, 0.0980], [0.4660, 0.6740,
128     0.1880]};
129
130 bar_handles = bar(total_costs, 'FaceColor', 'flat');
131 for i = 1:3
132     bar_handles.CData(i,:) = colors{i};
133 end
134
135 % 添加数值标签

```

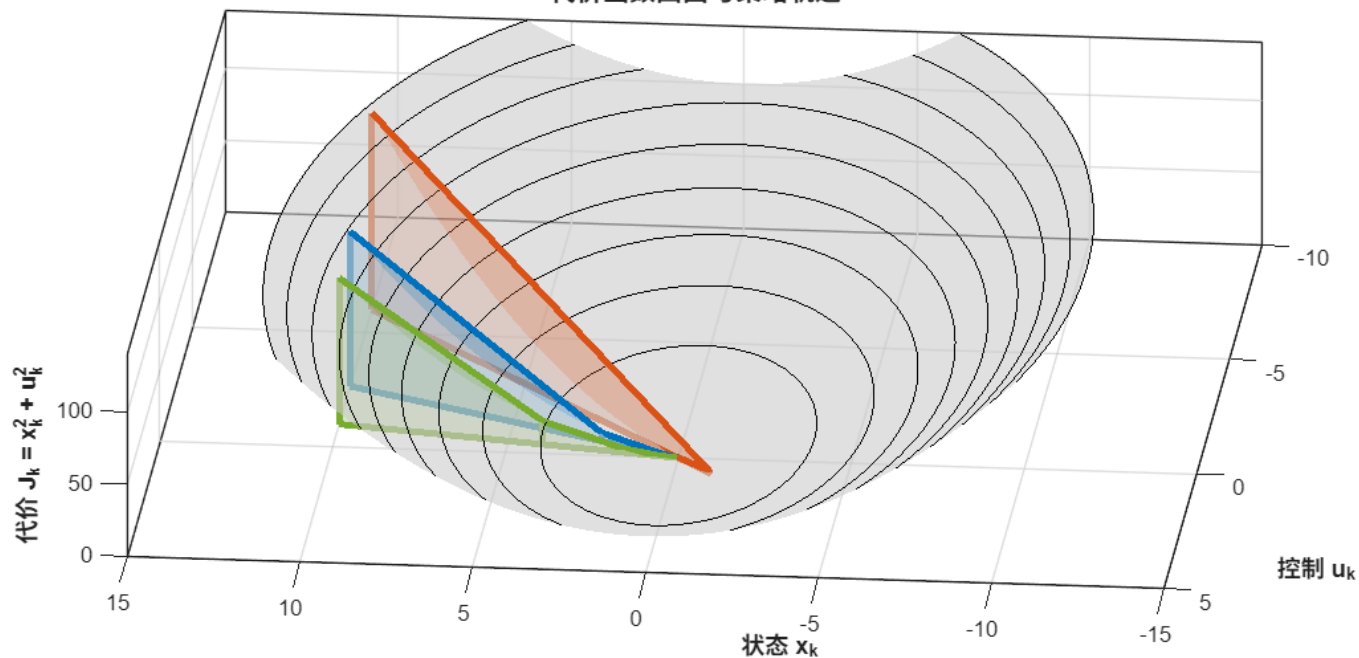
```

131     for i = 1:3
132         text(i, total_costs(i), sprintf('%.2f', total_costs(i)), ...
133             'HorizontalAlignment', 'center', 'VerticalAlignment', 'bottom', ...
134             'FontSize', 10, 'FontWeight', 'bold');
135     end
136
137     ylabel('总代价 J =  $\Sigma(x_k^2 + u_k^2)$ ', 'FontSize', 11, 'FontWeight', 'bold');
138     title('三种策略总代价比较', 'FontSize', 12, 'FontWeight', 'bold');
139     set(gca, 'XTickLabel', strategies, 'FontSize', 10);
140     grid on;
141
142     %% 绘制每个时刻的代价对比
143     ax3 = subplot(2,2,4);
144     hold on;
145     k_steps = 0:N-1;
146     plot(k_steps, J_opt, '-o', 'Color', [0, 0.4470, 0.7410], 'LineWidth', 2,
147         'MarkerFaceColor', [0, 0.4470, 0.7410], 'DisplayName', 'LQR最优');
147     plot(k_steps, J_strong, '-s', 'Color', [0.8500, 0.3250, 0.0980], 'LineWidth',
148         2, 'MarkerFaceColor', [0.8500, 0.3250, 0.0980], 'DisplayName', '较强反馈');
148     plot(k_steps, J_con, '-^', 'Color', [0.4660, 0.6740, 0.1880], 'LineWidth', 2,
149         'MarkerFaceColor', [0.4660, 0.6740, 0.1880], 'DisplayName', '保守控制');
149
150     xlabel('时刻 k', 'FontSize', 11, 'FontWeight', 'bold');
151     ylabel('瞬时代价 J_k =  $x_k^2 + u_k^2$ ', 'FontSize', 11, 'FontWeight', 'bold');
152     title('各时刻代价对比', 'FontSize', 12, 'FontWeight', 'bold');
153     legend('Location', 'best');
154     grid on;
155     xlim([-0.5, N-0.5]);
156
157     %% 优化布局
158     sgtitle('离散最优控制：代价函数分析与比较', 'FontSize', 16, 'FontWeight', 'bold');

```

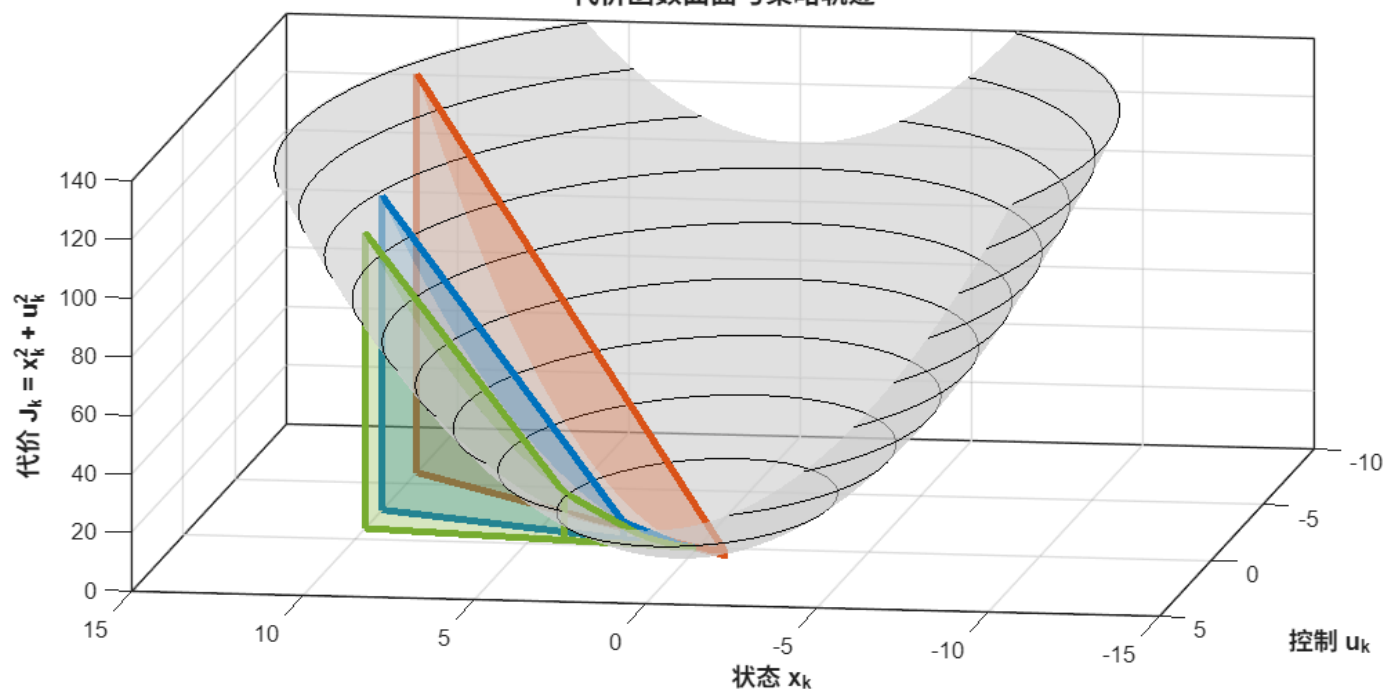
策略1: LQR最优
策略2: 较强反馈
策略3: 保守控制

代价函数曲面与策略轨迹



策略1: LQR最优
策略2: 较强反馈
策略3: 保守控制

代价函数曲面与策略轨迹



代码块

```
1 %% 第四步: 绘制  $x_{k+1}-x_k-J$  三维相图
2 clear; clc; close all;
3
4 % 系统参数
5 A = 0.5;
6 B = 1;
```

```

7  x0 = 10;
8  N = 8; % 仿真步数
9  Q = 1; % 状态代价权重
10 R = 1; % 控制代价权重
11
12 % 计算LQR最优增益
13 [K_opt, ~, ~] = dlqr(A, B, Q, R); % 最优反馈增益  $K_{opt} \approx 0.2655$ 
14
15 % 1. 策略1: LQR最优控制
16 x_opt = zeros(1, N+1);
17 u_opt = zeros(1, N);
18 J_opt = zeros(1, N); % 存储每个时刻的代价函数值
19 x_opt(1) = x0;
20
21 for k = 1:N
22     u_opt(k) = -K_opt * x_opt(k);
23     J_opt(k) = x_opt(k)^2 + u_opt(k)^2; % 当前时刻的代价
24     x_opt(k+1) = A * x_opt(k) + B * u_opt(k);
25 end
26
27 % 2. 策略2: 较强反馈控制 ( $u = -0.8x$ )
28 K_strong = 0.6;
29 x_strong = zeros(1, N+1);
30 u_strong = zeros(1, N);
31 J_strong = zeros(1, N);
32 x_strong(1) = x0;
33
34 for k = 1:N
35     u_strong(k) = -K_strong * x_strong(k);
36     J_strong(k) = x_strong(k)^2 + u_strong(k)^2;
37     x_strong(k+1) = A * x_strong(k) + B * u_strong(k);
38 end
39
40 % 3. 策略3: 保守控制 ( $u = -0.1x$ )
41 K_con = 0.1;
42 x_con = zeros(1, N+1);
43 u_con = zeros(1, N);
44 J_con = zeros(1, N);
45 x_con(1) = x0;
46
47 for k = 1:N
48     u_con(k) = -K_con * x_con(k);
49     J_con(k) = x_con(k)^2 + u_con(k)^2;
50     x_con(k+1) = A * x_con(k) + B * u_con(k);
51 end
52
53 %% 计算总代价

```



```

54 total_cost_opt = sum(J_opt);
55 total_cost_strong = sum(J_strong);
56 total_cost_con = sum(J_con);
57
58 %% 绘制  $x_{k+1}-x_k-J$  三维相图
59 figure('Color', 'w', 'Position', [100, 100, 1200, 900]);
60
61 % 创建三维坐标系
62 ax1 = subplot(2,2,[1,3]);
63 hold on; grid on; box on;
64 view(45, 30); % 设置视角
65
66 % 绘制三条轨迹
67 % 策略1: LQR最优 (蓝色)
68 for k = 1:N
69     plot3([x_opt(k), x_opt(k+1)], [x_opt(k), x_opt(k+1)], [0, 0], 'k:',
70 'LineWidth', 0.5); % 投影线
71 end
72 h1 = plot3(x_opt(1:N), x_opt(2:N+1), J_opt, '-o', 'Color', [0, 0.4470,
73 0.7410], ...
74 'LineWidth', 2, 'MarkerFaceColor', [0, 0.4470, 0.7410], 'MarkerSize', 8,
75 ...
76 'DisplayName', sprintf('策略1: LQR最优 (J=%.2f)', total_cost_opt));
77
78 % 策略2: 较强反馈 (红色)
79 for k = 1:N
80     plot3([x_strong(k), x_strong(k+1)], [x_strong(k), x_strong(k+1)], [0, 0],
81 'k:', 'LineWidth', 0.5); % 投影线
82 end
83 h2 = plot3(x_strong(1:N), x_strong(2:N+1), J_strong, '-s', 'Color', [0.8500,
84 0.3250, 0.0980], ...
85 'LineWidth', 2, 'MarkerFaceColor', [0.8500, 0.3250, 0.0980], 'MarkerSize',
86 8, ...
87 'DisplayName', sprintf('策略2: 较强反馈 (J=%.2f)', total_cost_strong));
88
89 % 策略3: 保守控制 (绿色)
90 for k = 1:N
91     plot3([x_con(k), x_con(k+1)], [x_con(k), x_con(k+1)], [0, 0], 'k:',
92 'LineWidth', 0.5); % 投影线
93 end
94 h3 = plot3(x_con(1:N), x_con(2:N+1), J_con, '-^', 'Color', [0.4660, 0.6740,
95 0.1880], ...
96 'LineWidth', 2, 'MarkerFaceColor', [0.4660, 0.6740, 0.1880], 'MarkerSize',
97 8, ...
98 'DisplayName', sprintf('策略3: 保守控制 (J=%.2f)', total_cost_con));
99
100 % 为每个点标注时刻k

```

```

92  for k = 1:min(5, N) % 标注前5个点
93      text(x_opt(k), x_opt(k+1), J_opt(k)+5, sprintf('k=%d→%d', k-1, k), ...
94          'FontSize', 9, 'HorizontalAlignment', 'center', 'Color', [0, 0.4470,
0.7410]);
95      text(x_strong(k), x_strong(k+1), J_strong(k)+5, sprintf('k=%d→%d', k-1,
k), ...
96          'FontSize', 9, 'HorizontalAlignment', 'center', 'Color', [0.8500,
0.3250, 0.0980]);
97      text(x_con(k), x_con(k+1), J_con(k)+5, sprintf('k=%d→%d', k-1, k), ...
98          'FontSize', 9, 'HorizontalAlignment', 'center', 'Color', [0.4660,
0.6740, 0.1880]);
99  end
100
101  % 绘制单位直线  $x_{k+1} = x_k$  (稳定边界)
102  x_range = linspace(-10, 10, 100);
103  plot3(x_range, x_range, zeros(size(x_range)), 'k--', 'LineWidth', 1.5, ...
104      'DisplayName', 'x_{k+1} = x_k (稳定边界)');
105
106  % 绘制吸引子线  $x_{k+1} = 0.5x_k$  (无控制时的系统动态)
107  plot3(x_range, 0.5*x_range, zeros(size(x_range)), 'm--', 'LineWidth', 1.5, ...
108      'DisplayName', 'x_{k+1} = 0.5x_k (开环系统)');
109
110  % 添加坐标轴标签
111  xlabel('当前状态 x_k', 'FontSize', 12, 'FontWeight', 'bold');
112  ylabel('下一状态 x_{k+1}', 'FontSize', 12, 'FontWeight', 'bold');
113  zlabel('瞬时代价 J_k = x_k^2 + u_k^2', 'FontSize', 12, 'FontWeight', 'bold');
114  title('x_{k+1}-x_k-J 三维相图: 状态转移与代价', 'FontSize', 14, 'FontWeight',
'bold');
115  legend('Location', 'best');
116  grid on;
117
118  % 设置坐标轴范围
119  xlim([-11, 11]);
120  ylim([-4, 11]);
121  zlim([0, max([J_opt, J_strong, J_con])*1.1]);
122
123  % 绘制x-y平面上的网格
124  [x_plane, y_plane] = meshgrid(-10:2:10, -3:2:10);
125  z_plane = zeros(size(x_plane));
126  surf(x_plane, y_plane, z_plane, 'FaceAlpha', 0.1, 'EdgeColor', [0.7, 0.7,
0.7], 'FaceColor', [0.9, 0.9, 0.9]);
127
128  %% 绘制状态转移平面图
129  ax2 = subplot(2,2,2);
130  hold on; grid on; box on;
131
132  % 绘制三条状态转移轨迹

```

```

133 plot(x_opt(1:N), x_opt(2:N+1), '-o', 'Color', [0, 0.4470, 0.7410], ...
134       'LineWidth', 2, 'MarkerFaceColor', [0, 0.4470, 0.7410], 'MarkerSize', 8,
135       ...
136       'DisplayName', '策略1: LQR最优');
137 plot(x_strong(1:N), x_strong(2:N+1), '-s', 'Color', [0.8500, 0.3250, 0.0980],
138       ...
139       'LineWidth', 2, 'MarkerFaceColor', [0.8500, 0.3250, 0.0980], 'MarkerSize',
140       8, ...
141       'DisplayName', '策略2: 较强反馈');
142 plot(x_con(1:N), x_con(2:N+1), '-^', 'Color', [0.4660, 0.6740, 0.1880], ...
143       'LineWidth', 2, 'MarkerFaceColor', [0.4660, 0.6740, 0.1880], 'MarkerSize',
144       8, ...
145       'DisplayName', '策略3: 保守控制');
146
147 % 标注初始和最终状态
148 plot(x0, x_opt(2), 'bo', 'MarkerSize', 12, 'LineWidth', 2, 'DisplayName', '初始
149 状态');
150 plot(0, 0, 'r*', 'MarkerSize', 15, 'LineWidth', 2, 'DisplayName', '目标状态(原
151 点)');
152
153 % 绘制参考线
154 plot(x_range, x_range, 'k--', 'LineWidth', 1.5, 'DisplayName', 'x_{k+1} =
155 x_k');
156 plot(x_range, 0.5*x_range, 'm--', 'LineWidth', 1.5, 'DisplayName', 'x_{k+1} =
157 0.5x_k');
158
159 % 为每个点标注时刻
160 for k = 1:min(4, N)
161     text(x_opt(k), x_opt(k+1), sprintf('k=%d→%d', k-1, k), ...
162          'FontSize', 9, 'HorizontalAlignment', 'center', 'VerticalAlignment',
163          'bottom', 'Color', [0, 0.4470, 0.7410]);
164     text(x_strong(k), x_strong(k+1), sprintf('k=%d→%d', k-1, k), ...
165          'FontSize', 9, 'HorizontalAlignment', 'center', 'VerticalAlignment',
166          'bottom', 'Color', [0.8500, 0.3250, 0.0980]);
167     text(x_con(k), x_con(k+1), sprintf('k=%d→%d', k-1, k), ...
168          'FontSize', 9, 'HorizontalAlignment', 'center', 'VerticalAlignment',
169          'bottom', 'Color', [0.4660, 0.6740, 0.1880]);
170 end
171
172 xlabel('当前状态 x_k', 'FontSize', 11, 'FontWeight', 'bold');
173 ylabel('下一状态 x_{k+1}', 'FontSize', 11, 'FontWeight', 'bold');
174 title('状态转移平面图', 'FontSize', 12, 'FontWeight', 'bold');
175 legend('Location', 'best');
176 xlim([-4, 11]);
177 ylim([-4, 6]);
178 axis equal;
179

```

```

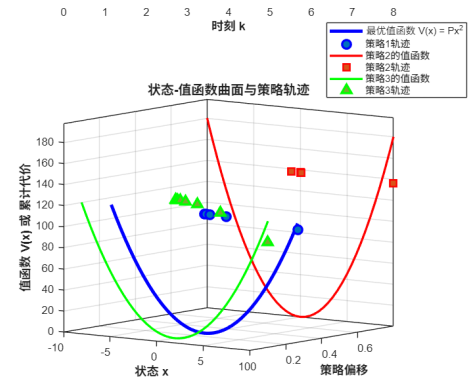
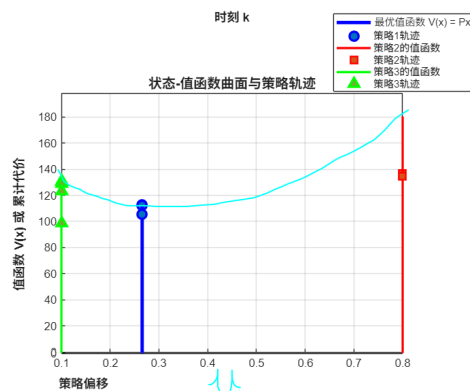
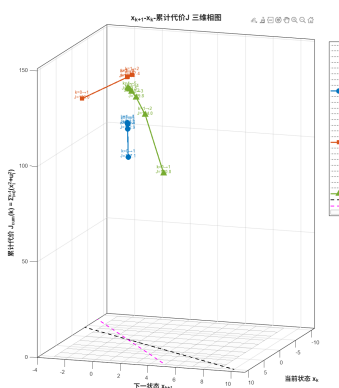
169 %% 绘制代价函数随状态变化的曲面
170 ax3 = subplot(2,2,4);
171 hold on; grid on; box on;
172 view(45, 30);
173
174 % 创建网格
175 [X, U] = meshgrid(-15:0.5:15, -10:0.5:5);
176 % 计算代价函数  $J = x^2 + u^2$ 
177 J_surface = X.^2 + U.^2;
178
179 % 绘制代价函数曲面
180 surf(X, U, J_surface, 'FaceAlpha', 0.6, 'EdgeColor', 'none', 'FaceColor',
    [0.8, 0.8, 0.8]);
181 colormap(ax3, 'parula');
182 % 绘制等高线
183 contour3(X, U, J_surface, 20, 'k', 'LineWidth', 0.5);
184
185 % 在曲面上绘制三条控制策略的轨迹
186 % 策略1: LQR最优
187 U1 = -K_opt * x_opt(1:N);
188 surf_x1 = [x_opt(1:N); x_opt(1:N)];
189 surf_u1 = [U1; U1];
190 surf_j1 = [zeros(1, N); J_opt];
191 h_s1 = mesh(surf_x1, surf_u1, surf_j1, 'FaceColor', [0, 0.4470, 0.7410],
    'EdgeColor', [0, 0.4470, 0.7410], 'LineWidth', 3, 'FaceAlpha', 0.3);
192
193 % 策略2: 较强反馈
194 U2 = -K_strong * x_strong(1:N);
195 surf_x2 = [x_strong(1:N); x_strong(1:N)];
196 surf_u2 = [U2; U2];
197 surf_j2 = [zeros(1, N); J_strong];
198 h_s2 = mesh(surf_x2, surf_u2, surf_j2, 'FaceColor', [0.8500, 0.3250, 0.0980],
    'EdgeColor', [0.8500, 0.3250, 0.0980], 'LineWidth', 3, 'FaceAlpha', 0.3);
199
200 % 策略3: 保守控制
201 U3 = -K_con * x_con(1:N);
202 surf_x3 = [x_con(1:N); x_con(1:N)];
203 surf_u3 = [U3; U3];
204 surf_j3 = [zeros(1, N); J_con];
205 h_s3 = mesh(surf_x3, surf_u3, surf_j3, 'FaceColor', [0.4660, 0.6740, 0.1880],
    'EdgeColor', [0.4660, 0.6740, 0.1880], 'LineWidth', 3, 'FaceAlpha', 0.3);
206
207 xlabel('状态 x_k', 'FontSize', 11, 'FontWeight', 'bold');
208 ylabel('控制 u_k', 'FontSize', 11, 'FontWeight', 'bold');
209 zlabel('代价 J_k = x_k^2 + u_k^2', 'FontSize', 11, 'FontWeight', 'bold');
210 title('代价函数曲面与策略轨迹', 'FontSize', 12, 'FontWeight', 'bold');
211 zlim([0, 140]);

```

```

212
213 % 创建自定义图例
214 hL = legend([h_s1, h_s2, h_s3], {'策略1: LQR最优', '策略2: 较强反馈', '策略3: 保守控制'}, 'Location', 'northeast');
215
216 %% 添加总代价比较表格
217 % annotation('textbox', [0.15, 0.05, 0.7, 0.1], 'String', ...
218 %     sprintf(['总代价比较 (N=%d步):\n', ...
219 %         '策略1 (LQR最优, K=%4f): J = %.4f\n', ...
220 %         '策略2 (较强反馈, K=0.8): J = %.4f\n', ...
221 %         '策略3 (保守控制, K=0.1): J = %.4f\n\n', ...
222 %         '最优性结论: LQR策略在给定代价函数 J = \Sigma(x_k^2 + u_k^2) 下是最优的'], ...
223 %     N, K_opt, total_cost_opt, total_cost_strong, total_cost_con), ...
224 %     'FontSize', 10, 'FontName', 'Monospace', ...
225 %     'BackgroundColor', [0.95, 0.95, 0.95], 'EdgeColor', 'k', ...
226 %     'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'center');
227
228 %% 优化布局
229 sgtitle('离散系统状态转移与代价函数分析', 'FontSize', 16, 'FontWeight', 'bold');

```



上面的最后几张图还有些许问题，可以先跳过。

代码块

```

1 %% 绘制 x_{k+1}-x_k-累计代价J 三维相图
2 clear; clc; close all;
3
4 % 系统参数
5 A = 0.5;
6 B = 1;
7 x0 = 10;
8 N = 8; % 仿真步数
9 Q = 1; % 状态代价权重
10 R = 1; % 控制代价权重
11
12 % 计算LQR最优增益
13 [K_opt, ~, ~] = dlqr(A, B, Q, R); % 最优反馈增益 K_opt ≈ 0.2655

```

```

14
15 % 1. 策略1: LQR最优控制
16 x_opt = zeros(1, N+1);
17 u_opt = zeros(1, N);
18 J_opt_single = zeros(1, N); % 存储每个时刻的单步代价
19 J_opt_cum = zeros(1, N+1); % 存储累计代价, J_opt_cum(1)=0
20 x_opt(1) = x0;
21 J_opt_cum(1) = 0; % 初始累计代价为0
22
23 for k = 1:N
24     u_opt(k) = -K_opt * x_opt(k);
25     J_opt_single(k) = x_opt(k)^2 + u_opt(k)^2; % 当前时刻的单步代价
26     J_opt_cum(k+1) = J_opt_cum(k) + J_opt_single(k); % 更新累计代价
27     x_opt(k+1) = A * x_opt(k) + B * u_opt(k);
28 end
29
30 % 2. 策略2: 较强反馈控制 ( $u = -0.8x$ )
31 K_strong = 0.6;
32 x_strong = zeros(1, N+1);
33 u_strong = zeros(1, N);
34 J_strong_single = zeros(1, N);
35 J_strong_cum = zeros(1, N+1);
36 x_strong(1) = x0;
37 J_strong_cum(1) = 0;
38
39 for k = 1:N
40     u_strong(k) = -K_strong * x_strong(k);
41     J_strong_single(k) = x_strong(k)^2 + u_strong(k)^2;
42     J_strong_cum(k+1) = J_strong_cum(k) + J_strong_single(k);
43     x_strong(k+1) = A * x_strong(k) + B * u_strong(k);
44 end
45
46 % 3. 策略3: 保守控制 ( $u = -0.1x$ )
47 K_con = 0.01;
48 x_con = zeros(1, N+1);
49 u_con = zeros(1, N);
50 J_con_single = zeros(1, N);
51 J_con_cum = zeros(1, N+1);
52 x_con(1) = x0;
53 J_con_cum(1) = 0;
54
55 for k = 1:N
56     u_con(k) = -K_con * x_con(k);
57     J_con_single(k) = x_con(k)^2 + u_con(k)^2;
58     J_con_cum(k+1) = J_con_cum(k) + J_con_single(k);
59     x_con(k+1) = A * x_con(k) + B * u_con(k);
60 end

```

```

61
62 % 计算最终总代价
63 total_cost_opt = J_opt_cum(end);
64 total_cost_strong = J_strong_cum(end);
65 total_cost_con = J_con_cum(end);
66
67 %% 绘制  $x_{k+1}$ - $x_k$ -累计代价J 三维相图
68 figure('Color', 'w', 'Position', [100, 100, 1200, 900]);
69
70 % 创建三维坐标系
71 ax1 = subplot(2,2,[1,3]);
72 hold on; grid on; box on;
73 view(45, 30); % 设置视角
74
75 % 绘制三条轨迹
76 % 策略1: LQR最优 (蓝色)
77 for k = 1:N
78     % 绘制状态转移线和累计代价柱
79     plot3([x_opt(k), x_opt(k+1)], [x_opt(k), x_opt(k+1)], [0, 0], 'k:',
80 'LineWidth', 0.5); % 投影线
81 end
82 h1 = plot3(x_opt(1:N), x_opt(2:N+1), J_opt_cum(2:end), '-o', 'Color', [0,
83 0.4470, 0.7410], ...
84 'LineWidth', 2, 'MarkerFaceColor', [0, 0.4470, 0.7410], 'MarkerSize', 8,
85 ...
86 'DisplayName', sprintf('策略1: LQR最优 (总J=%.2f)', total_cost_opt));
87
88 % 策略2: 较强反馈 (红色)
89 for k = 1:N
90     plot3([x_strong(k), x_strong(k+1)], [x_strong(k), x_strong(k+1)], [0, 0],
91 'k:', 'LineWidth', 0.5);
92 end
93 h2 = plot3(x_strong(1:N), x_strong(2:N+1), J_strong_cum(2:end), '-s', 'Color',
94 [0.8500, 0.3250, 0.0980], ...
95 'LineWidth', 2, 'MarkerFaceColor', [0.8500, 0.3250, 0.0980], 'MarkerSize',
96 8, ...
97 'DisplayName', sprintf('策略2: 较强反馈 (总J=%.2f)', total_cost_strong));
98
99 % 策略3: 保守控制 (绿色)
100 for k = 1:N
101     plot3([x_con(k), x_con(k+1)], [x_con(k), x_con(k+1)], [0, 0], 'k:',
102 'LineWidth', 0.5);
103 end
104 h3 = plot3(x_con(1:N), x_con(2:N+1), J_con_cum(2:end), '-^', 'Color', [0.4660,
105 0.6740, 0.1880], ...
106 'LineWidth', 2, 'MarkerFaceColor', [0.4660, 0.6740, 0.1880], 'MarkerSize',
107 8, ...

```

```

99     'DisplayName', sprintf('策略3: 保守控制 (总J=%.2f)', total_cost_con));
100
101 % 为每个点标注时刻k和累计代价
102 for k = 1:min(5, N) % 标注前5个点
103     text(x_opt(k), x_opt(k+1), J_opt_cum(k+1)+2, ...
104          sprintf('k=%d→%d\nJ=%.1f', k-1, k, J_opt_cum(k+1)), ...
105          'FontSize', 8, 'HorizontalAlignment', 'center', 'Color', [0, 0.4470,
0.7410]);
106     text(x_strong(k), x_strong(k+1), J_strong_cum(k+1)+2, ...
107          sprintf('k=%d→%d\nJ=%.1f', k-1, k, J_strong_cum(k+1)), ...
108          'FontSize', 8, 'HorizontalAlignment', 'center', 'Color', [0.8500,
0.3250, 0.0980]);
109     text(x_con(k), x_con(k+1), J_con_cum(k+1)+2, ...
110          sprintf('k=%d→%d\nJ=%.1f', k-1, k, J_con_cum(k+1)), ...
111          'FontSize', 8, 'HorizontalAlignment', 'center', 'Color', [0.4660,
0.6740, 0.1880]);
112 end
113
114 % 绘制单位直线  $x_{k+1} = x_k$  (稳定边界)
115 x_range = linspace(-10, 10, 100);
116 plot3(x_range, x_range, zeros(size(x_range)), 'k--', 'LineWidth', 1.5, ...
117       'DisplayName', 'x_{k+1} = x_k (稳定边界)');
118
119 % 绘制吸引子线  $x_{k+1} = 0.5x_k$  (无控制时的系统动态)
120 plot3(x_range, 0.5*x_range, zeros(size(x_range)), 'm--', 'LineWidth', 1.5, ...
121       'DisplayName', 'x_{k+1} = 0.5x_k (开环系统)');
122
123 % 添加坐标轴标签
124 xlabel('当前状态 x_k', 'FontSize', 12, 'FontWeight', 'bold');
125 ylabel('下一状态 x_{k+1}', 'FontSize', 12, 'FontWeight', 'bold');
126 zlabel('累计代价 J_{cum}(k) = \sum_{i=0}^{k-1}(x_i^2+u_i^2)', 'FontSize', 12,
'FontWeight', 'bold');
127 title('x_{k+1}-x_k-累计代价J 三维相图', 'FontSize', 14, 'FontWeight', 'bold');
128 legend('Location', 'best');
129 grid on;
130
131 % 设置坐标轴范围
132 xlim([-11, 11]);
133 ylim([-4, 11]);
134 zlim([0, max([J_opt_cum, J_strong_cum, J_con_cum])*1.1]);
135
136 % 绘制x-y平面上的网格
137 [x_plane, y_plane] = meshgrid(-10:2:10, -3:2:10);
138 z_plane = zeros(size(x_plane));
139 surf(x_plane, y_plane, z_plane, 'FaceAlpha', 0.1, 'EdgeColor', [0.7, 0.7,
0.7], 'FaceColor', [0.9, 0.9, 0.9]);
140

```



```

141 %% 绘制累计代价随时间变化曲线
142 ax2 = subplot(2,2,2);
143 hold on; grid on; box on;
144
145 % 绘制三条累计代价曲线
146 k_steps = 0:N; % 包括k=0时刻
147 plot(k_steps, J_opt_cum, '-o', 'Color', [0, 0.4470, 0.7410], ...
148      'LineWidth', 2, 'MarkerFaceColor', [0, 0.4470, 0.7410], 'MarkerSize', 8,
149      'DisplayName', '策略1: LQR最优');
150 plot(k_steps, J_strong_cum, '-s', 'Color', [0.8500, 0.3250, 0.0980], ...
151      'LineWidth', 2, 'MarkerFaceColor', [0.8500, 0.3250, 0.0980], 'MarkerSize',
152      8, ...
153      'DisplayName', '策略2: 较强反馈');
154 plot(k_steps, J_con_cum, '-^', 'Color', [0.4660, 0.6740, 0.1880], ...
155      'LineWidth', 2, 'MarkerFaceColor', [0.4660, 0.6740, 0.1880], 'MarkerSize',
156      8, ...
157      'DisplayName', '策略3: 保守控制');
158
159 % 标注最终总代价
160 text(N, J_opt_cum(end), sprintf('%.1f', J_opt_cum(end)), ...
161      'FontSize', 10, 'HorizontalAlignment', 'left', 'VerticalAlignment',
162      'middle', 'Color', [0, 0.4470, 0.7410]);
163 text(N, J_strong_cum(end), sprintf('%.1f', J_strong_cum(end)), ...
164      'FontSize', 10, 'HorizontalAlignment', 'left', 'VerticalAlignment',
165      'middle', 'Color', [0.8500, 0.3250, 0.0980]);
166 text(N, J_con_cum(end), sprintf('%.1f', J_con_cum(end)), ...
167      'FontSize', 10, 'HorizontalAlignment', 'left', 'VerticalAlignment',
168      'middle', 'Color', [0.4660, 0.6740, 0.1880]);
169
170 xlabel('时刻 k', 'FontSize', 11, 'FontWeight', 'bold');
171 ylabel('累计代价 J_{cum}(k)', 'FontSize', 11, 'FontWeight', 'bold');
172 title('累计代价随时间变化', 'FontSize', 12, 'FontWeight', 'bold');
173 legend('Location', 'northwest');
174 xlim([0, N]);
175 ylim([0, max([J_opt_cum, J_strong_cum, J_con_cum])*1.1]);
176
177 %% 绘制状态-累计代价曲面
178 ax3 = subplot(2,2,4);
179 hold on; grid on; box on;
180 view(45, 30);
181
182 % 创建状态网格
183 X = -10:0.5:10;
184
185 % 对于每个状态x, 计算在LQR最优控制下的最小累计代价
186 % 这里我们计算从状态x到原点的最小代价函数值 (值函数V(x))
187 % 对于LQR问题, 值函数为  $V(x) = P * x^2$ , 其中P是Riccati方程的解

```

```

182 % 计算Riccati方程的解P
183 [~, P_opt, ~] = dlqr(A, B, Q, R);
184 V_opt = P_opt * (X.^2);
185
186 % 绘制值函数曲面
187 plot3(X, ones(size(X))*K_opt, V_opt, 'b-', 'LineWidth', 3, 'DisplayName', '最优
值函数 V(x) = Px^2');
188
189 % 在值函数曲面上绘制三个策略的轨迹
190 % 策略1: LQR最优
191 plot3(x_opt(1:N), ones(1, N)*K_opt, J_opt_cum(2:end), 'bo', 'LineWidth', 2, ...
192       'MarkerFaceColor', [0, 0.4470, 0.7410], 'MarkerSize', 8, 'DisplayName', '策略1轨迹');
193
194 % 策略2: 较强反馈
195 % 计算这个策略对应的"值函数" (虽然不是最优的)
196 K2 = 0.8;
197 A_cl2 = A - B*K2; % 闭环系统矩阵
198 % 计算这个非最优策略对应的代价 (通过求解Lyapunov方程)
199 P2 = dlyap(A_cl2', Q + K2'*R*K2);
200 V_strong = P2 * (X.^2);
201 plot3(X, ones(size(X))*K2, V_strong, 'r-', 'LineWidth', 2, 'DisplayName', '策略2
的值函数');
202
203 plot3(x_strong(1:N), ones(1, N)*K2, J_strong_cum(2:end), 'rs', 'LineWidth', 2,
...
204       'MarkerFaceColor', [0.8500, 0.3250, 0.0980], 'MarkerSize', 8,
'DisplayName', '策略2轨迹');
205
206 % 策略3: 保守控制
207 K3 = 0.1;
208 A_cl3 = A - B*K3;
209 P3 = dlyap(A_cl3', Q + K3'*R*K3);
210 V_con = P3 * (X.^2);
211 plot3(X, ones(size(X))*K3, V_con, 'g-', 'LineWidth', 2, 'DisplayName', '策略3的
值函数');
212
213 plot3(x_con(1:N), ones(1, N)*K3, J_con_cum(2:end), 'g^', 'LineWidth', 2, ...
214       'MarkerFaceColor', [0.4660, 0.6740, 0.1880], 'MarkerSize', 8,
'DisplayName', '策略3轨迹');
215
216 xlabel('状态 x', 'FontSize', 11, 'FontWeight', 'bold');
217 ylabel('策略偏移', 'FontSize', 11, 'FontWeight', 'bold');
218 zlabel('值函数 V(x) 或 累计代价', 'FontSize', 11, 'FontWeight', 'bold');
219 title('状态-值函数曲面与策略轨迹', 'FontSize', 12, 'FontWeight', 'bold');
220 legend('Location', 'northwest');
221 xlim([-10, 10]);

```

```

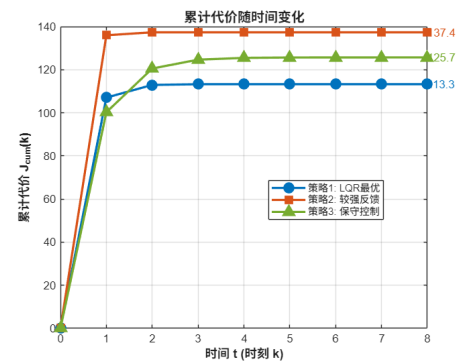
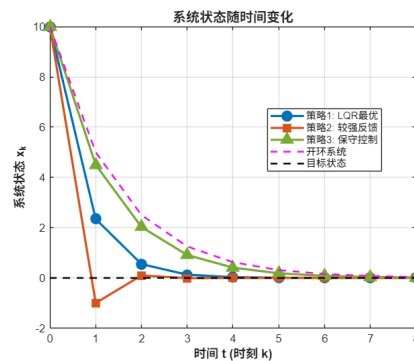
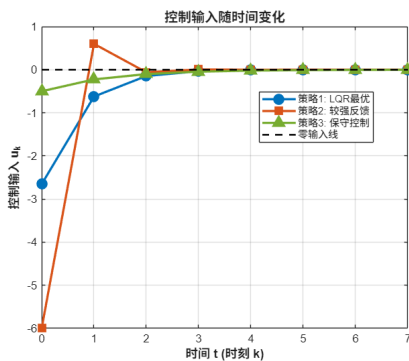
222 zlim([0, max([V_opt, V_strong, V_con, J_opt_cum(end)])*1.1]);
223
224 % 添加网格
225 grid on;
226
227 %% 绘制单步代价对比
228 figure('Color', 'w', 'Position', [200, 200, 600, 500]);
229 subplot(2,1,1);
230 hold on; grid on;
231 k_steps_single = 0:N-1;
232 plot(k_steps_single, J_opt_single, '-o', 'Color', [0, 0.4470, 0.7410], ...
233      'LineWidth', 2, 'MarkerFaceColor', [0, 0.4470, 0.7410], 'MarkerSize', 8,
234      ...
235      'DisplayName', '策略1: LQR最优');
236 plot(k_steps_single, J_strong_single, '-s', 'Color', [0.8500, 0.3250, 0.0980],
237      ...
238      'LineWidth', 2, 'MarkerFaceColor', [0.8500, 0.3250, 0.0980], 'MarkerSize',
239      8, ...
240      'DisplayName', '策略2: 较强反馈');
241 plot(k_steps_single, J_con_single, '-^', 'Color', [0.4660, 0.6740, 0.1880], ...
242      'LineWidth', 2, 'MarkerFaceColor', [0.4660, 0.6740, 0.1880], 'MarkerSize',
243      8, ...
244      'DisplayName', '策略3: 保守控制');
245 xlabel('时刻 k', 'FontSize', 11, 'FontWeight', 'bold');
246 ylabel('单步代价 J_k = x_k^2 + u_k^2', 'FontSize', 11, 'FontWeight', 'bold');
247 title('单步代价对比', 'FontSize', 12, 'FontWeight', 'bold');
248 legend('Location', 'northeast');
249 xlim([0, N-1]);
250
251 % 绘制总代价比较
252 subplot(2,1,2);
253 hold on; grid on;
254 strategies = {'LQR最优', '较强反馈', '保守控制'};
255 total_costs = [total_cost_opt, total_cost_strong, total_cost_con];
256 colors = {[0, 0.4470, 0.7410], [0.8500, 0.3250, 0.0980], [0.4660, 0.6740,
257 0.1880]};
258
259 bar_handles = bar(total_costs, 'FaceColor', 'flat');
260 for i = 1:3
261     bar_handles.CData(i,:) = colors{i};
262 end
263
264 % 添加数值标签
265 for i = 1:3
266     text(i, total_costs(i), sprintf('%.2f', total_costs(i)), ...
267          'HorizontalAlignment', 'center', 'VerticalAlignment', 'bottom', ...

```

```

264         'FontSize', 10, 'FontWeight', 'bold');
265     end
266
267     ylabel('总累计代价  $J = \sum_{k=0}^{N-1} (x_k^2 + u_k^2)$ ', 'FontSize', 11,
        'FontWeight', 'bold');
268     title('三种策略总累计代价比较', 'FontSize', 12, 'FontWeight', 'bold');
269     set(gca, 'XTickLabel', strategies, 'FontSize', 10);
270
271     %% 添加总结信息
272     annotation('textbox', [0.15, 0.02, 0.7, 0.05], 'String', ...
273         sprintf(['LQR最优性验证: 策略1 (LQR, K=%.4f) 的总累计代价最小 (%.4f) < 策略2 (%.4f) < 策略3 (%.4f)'], ...
274             K_opt, total_cost_opt, total_cost_strong, total_cost_con), ...
275         'FontSize', 10, 'FontName', 'Monospace', ...
276         'BackgroundColor', [0.95, 0.95, 0.95], 'EdgeColor', 'k', ...
277         'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'center');
278
279     sgtitle('离散系统状态转移与累计代价函数分析', 'FontSize', 16, 'FontWeight',
        'bold');

```



第一张图的三维图 沿着 u 方向画切面，可以看到 除了0的位置，其他的位置，蓝色线均在抛物线的最低点。说明了蓝线的轨迹选择每次都是累计价值的最优。第0个点（初始值）为不可控的，并且调整有一定的惯性，所以刚开始的时候蓝线不一定是最低点。

代码块

```

1  %% 绘制控制输入u和状态x随时间变化的三维图，z轴为累计代价
2  clear; clc; close all;
3
4  % 系统参数
5  A = 0.5;
6  B = 1;
7  x0 = 10;
8  N = 8; % 仿真步数
9  Q = 1; % 状态代价权重
10 R = 1; % 控制代价权重

```

```

11
12 % 计算LQR最优增益
13 [K_opt, ~, ~] = dlqr(A, B, Q, R); % 最优反馈增益  $K_{opt} \approx 0.2655$ 
14
15 % 策略定义
16 K_strong = 0.6; % 较强反馈增益
17 K_con = 0.05; % 保守反馈增益
18
19 % 1. 策略1: LQR最优控制
20 x_opt = zeros(1, N+1);
21 u_opt = zeros(1, N);
22 J_opt_single = zeros(1, N);
23 J_opt_cum = zeros(1, N+1);
24 x_opt(1) = x0;
25 J_opt_cum(1) = 0;
26
27 for k = 1:N
28     u_opt(k) = -K_opt * x_opt(k);
29     J_opt_single(k) = x_opt(k)^2 + u_opt(k)^2;
30     J_opt_cum(k+1) = J_opt_cum(k) + J_opt_single(k);
31     x_opt(k+1) = A * x_opt(k) + B * u_opt(k);
32 end
33
34 % 2. 策略2: 较强反馈控制
35 x_strong = zeros(1, N+1);
36 u_strong = zeros(1, N);
37 J_strong_single = zeros(1, N);
38 J_strong_cum = zeros(1, N+1);
39 x_strong(1) = x0;
40 J_strong_cum(1) = 0;
41
42 for k = 1:N
43     u_strong(k) = -K_strong * x_strong(k);
44     J_strong_single(k) = x_strong(k)^2 + u_strong(k)^2;
45     J_strong_cum(k+1) = J_strong_cum(k) + J_strong_single(k);
46     x_strong(k+1) = A * x_strong(k) + B * u_strong(k);
47 end
48
49 % 3. 策略3: 保守控制
50 x_con = zeros(1, N+1);
51 u_con = zeros(1, N);
52 J_con_single = zeros(1, N);
53 J_con_cum = zeros(1, N+1);
54 x_con(1) = x0;
55 J_con_cum(1) = 0;
56
57 for k = 1:N

```

```

58     u_con(k) = -K_con * x_con(k);
59     J_con_single(k) = x_con(k)^2 + u_con(k)^2;
60     J_con_cum(k+1) = J_con_cum(k) + J_con_single(k);
61     x_con(k+1) = A * x_con(k) + B * u_con(k);
62 end
63
64 % 计算最终总代价
65 total_cost_opt = J_opt_cum(end);
66 total_cost_strong = J_strong_cum(end);
67 total_cost_con = J_con_cum(end);
68
69 %% 图1: 控制输入u随时间变化, z轴为累计代价
70 figure('Color', 'w', 'Position', [100, 100, 1200, 900]);
71
72 % 创建三维坐标系
73 ax1 = subplot(2,2,[1,3]);
74 hold on; grid on; box on;
75 view(45, 30);
76
77 % 时间向量
78 t_u = 0:N-1; % 控制输入u的时间点
79 t_x = 0:N;    % 状态x的时间点
80
81 % 绘制三条控制输入轨迹
82 % 策略1: LQR最优 (蓝色)
83 plot3(t_u, u_opt, J_opt_cum(2:end), '-o', 'Color', [0, 0.4470, 0.7410], ...
84        'LineWidth', 2, 'MarkerFaceColor', [0, 0.4470, 0.7410], 'MarkerSize', 8,
85        ...
86        'DisplayName', sprintf('策略1: LQR最优 (K=%.3f, J=%.2f)', K_opt,
87        total_cost_opt));
88
89 % 策略2: 较强反馈 (红色)
90 plot3(t_u, u_strong, J_strong_cum(2:end), '-s', 'Color', [0.8500, 0.3250,
91        0.0980], ...
92        'LineWidth', 2, 'MarkerFaceColor', [0.8500, 0.3250, 0.0980], 'MarkerSize',
93        8, ...
94        'DisplayName', sprintf('策略2: 较强反馈 (K=%.3f, J=%.2f)', K_strong,
95        total_cost_strong));
96
97 % 策略3: 保守控制 (绿色)
98 plot3(t_u, u_con, J_con_cum(2:end), '-^', 'Color', [0.4660, 0.6740, 0.1880],
99        ...
100       'LineWidth', 2, 'MarkerFaceColor', [0.4660, 0.6740, 0.1880], 'MarkerSize',
101       8, ...
102       'DisplayName', sprintf('策略3: 保守控制 (K=%.3f, J=%.2f)', K_con,
103       total_cost_con));

```

```

97 % 为每个点标注时刻和累计代价
98 for k = 1:min(5, N)
99     text(t_u(k), u_opt(k), J_opt_cum(k+1)+3, ...
100         sprintf('k=%d\nu=%.2f\nJ=%.1f', t_u(k), u_opt(k), J_opt_cum(k+1)), ...
101         'FontSize', 8, 'HorizontalAlignment', 'center', 'Color', [0, 0.4470,
0.7410]);
102     text(t_u(k), u_strong(k), J_strong_cum(k+1)+3, ...
103         sprintf('k=%d\nu=%.2f\nJ=%.1f', t_u(k), u_strong(k),
J_strong_cum(k+1)), ...
104         'FontSize', 8, 'HorizontalAlignment', 'center', 'Color', [0.8500,
0.3250, 0.0980]);
105     text(t_u(k), u_con(k), J_con_cum(k+1)+3, ...
106         sprintf('k=%d\nu=%.2f\nJ=%.1f', t_u(k), u_con(k), J_con_cum(k+1)), ...
107         'FontSize', 8, 'HorizontalAlignment', 'center', 'Color', [0.4660,
0.6740, 0.1880]);
108 end
109
110 % 绘制零输入线
111 plot3([0, N-1], [0, 0], [0, 0], 'k--', 'LineWidth', 1.5, ...
112     'DisplayName', '零输入线 (u=0)');
113
114 % 添加坐标轴标签
115 xlabel('时间 t (时刻 k)', 'FontSize', 12, 'FontWeight', 'bold');
116 ylabel('控制输入 u_k', 'FontSize', 12, 'FontWeight', 'bold');
117 zlabel('累计代价 J_{cum}(k) = \sum_{i=0}^{k-1} (x_i^2 + u_i^2)', 'FontSize', 12,
'FontWeight', 'bold');
118 title('控制输入u随时间变化, z轴为累计代价', 'FontSize', 14, 'FontWeight', 'bold');
119 legend('Location', 'best');
120 grid on;
121
122 % 设置坐标轴范围
123 xlim([0, N-1]);
124 ylim([min([u_opt, u_strong, u_con])*1.1, max([u_opt, u_strong, u_con])*1.1]);
125 zlim([0, max([J_opt_cum, J_strong_cum, J_con_cum])*1.1]);
126
127 % 绘制投影平面
128 [t_grid, u_grid] = meshgrid(0:N-1, linspace(min([u_opt, u_strong, u_con]),
max([u_opt, u_strong, u_con]), 10));
129 z_grid = zeros(size(t_grid));
130 surf(t_grid, u_grid, z_grid, 'FaceAlpha', 0.1, 'EdgeColor', [0.7, 0.7, 0.7],
'FaceColor', [0.9, 0.9, 0.9]);
131
132 %% 图2: 状态x随时间变化, z轴为累计代价
133 ax2 = subplot(2,2,[2,4]);
134 hold on; grid on; box on;
135 view(45, 30);
136

```

```

137 % 绘制三条状态轨迹
138 % 策略1: LQR最优 (蓝色)
139 plot3(t_x, x_opt, J_opt_cum, '-o', 'Color', [0, 0.4470, 0.7410], ...
140     'LineWidth', 2, 'MarkerFaceColor', [0, 0.4470, 0.7410], 'MarkerSize', 8,
141     ...
142     'DisplayName', sprintf('策略1: LQR最优 (K=%.3f, J=%.2f)', K_opt,
total_cost_opt));
143
144 % 策略2: 较强反馈 (红色)
145 plot3(t_x, x_strong, J_strong_cum, '-s', 'Color', [0.8500, 0.3250, 0.0980], ...
146     'LineWidth', 2, 'MarkerFaceColor', [0.8500, 0.3250, 0.0980], 'MarkerSize',
8, ...
147     'DisplayName', sprintf('策略2: 较强反馈 (K=%.3f, J=%.2f)', K_strong,
total_cost_strong));
148
149 % 策略3: 保守控制 (绿色)
150 plot3(t_x, x_con, J_con_cum, '-^', 'Color', [0.4660, 0.6740, 0.1880], ...
151     'LineWidth', 2, 'MarkerFaceColor', [0.4660, 0.6740, 0.1880], 'MarkerSize',
8, ...
152     'DisplayName', sprintf('策略3: 保守控制 (K=%.3f, J=%.2f)', K_con,
total_cost_con));
153
154 % 为每个点标注时刻和累计代价
155 for k = 1:min(5, N+1)
156     text(t_x(k), x_opt(k), J_opt_cum(k)+2, ...
157         sprintf('k=%d\nx=%.2f\nJ=%.1f', t_x(k), x_opt(k), J_opt_cum(k)), ...
158         'FontSize', 8, 'HorizontalAlignment', 'center', 'Color', [0, 0.4470,
0.7410]);
159     text(t_x(k), x_strong(k), J_strong_cum(k)+2, ...
160         sprintf('k=%d\nx=%.2f\nJ=%.1f', t_x(k), x_strong(k), J_strong_cum(k)),
...
161         'FontSize', 8, 'HorizontalAlignment', 'center', 'Color', [0.8500,
0.3250, 0.0980]);
162     text(t_x(k), x_con(k), J_con_cum(k)+2, ...
163         sprintf('k=%d\nx=%.2f\nJ=%.1f', t_x(k), x_con(k), J_con_cum(k)), ...
164         'FontSize', 8, 'HorizontalAlignment', 'center', 'Color', [0.4660,
0.6740, 0.1880]);
165 end
166
167 % 绘制目标状态线
168 plot3([0, N], [0, 0], [0, 0], 'k--', 'LineWidth', 1.5, ...
169     'DisplayName', '目标状态 (x=0)');
170
171 % 绘制开环系统响应 (无控制)
172 x_open = x0 * (0.5).^(0:N);
173 plot3(t_x, x_open, zeros(1, N+1), 'm--', 'LineWidth', 1.5, ...
174     'DisplayName', '开环系统 (无控制)');

```



```

174
175 % 添加坐标轴标签
176 xlabel('时间 t (时刻 k)', 'FontSize', 12, 'FontWeight', 'bold');
177 ylabel('系统状态 x_k', 'FontSize', 12, 'FontWeight', 'bold');
178 zlabel('累计代价 J_{cum}(k) = \sum_{i=0}^{k-1}(x_i^2+u_i^2)', 'FontSize', 12,
'FontWeight', 'bold');
179 title('系统状态x随时间变化, z轴为累计代价', 'FontSize', 14, 'FontWeight', 'bold');
180 legend('Location', 'best');
181 grid on;
182
183 % 设置坐标轴范围
184 xlim([0, N]);
185 ylim([min([x_opt, x_strong, x_con])*1.1, max([x_opt, x_strong, x_con])*1.1]);
186 zlim([0, max([J_opt_cum, J_strong_cum, J_con_cum])*1.1]);
187
188 % 绘制投影平面
189 [t_grid, x_grid] = meshgrid(0:N, linspace(min([x_opt, x_strong, x_con]),
max([x_opt, x_strong, x_con]), 10));
190 z_grid = zeros(size(t_grid));
191 surf(t_grid, x_grid, z_grid, 'FaceAlpha', 0.1, 'EdgeColor', [0.7, 0.7, 0.7],
'FaceColor', [0.9, 0.9, 0.9]);
192
193 %% 绘制二维投影对比图
194 figure('Color', 'w', 'Position', [200, 200, 1200, 500]);
195
196 % 控制输入随时间变化
197 subplot(1,3,1);
198 hold on; grid on; box on;
199 plot(t_u, u_opt, '-o', 'Color', [0, 0.4470, 0.7410], ...
200      'LineWidth', 2, 'MarkerFaceColor', [0, 0.4470, 0.7410], 'MarkerSize', 8,
...
201      'DisplayName', '策略1: LQR最优');
202 plot(t_u, u_strong, '-s', 'Color', [0.8500, 0.3250, 0.0980], ...
203      'LineWidth', 2, 'MarkerFaceColor', [0.8500, 0.3250, 0.0980], 'MarkerSize',
8, ...
204      'DisplayName', '策略2: 较强反馈');
205 plot(t_u, u_con, '-^', 'Color', [0.4660, 0.6740, 0.1880], ...
206      'LineWidth', 2, 'MarkerFaceColor', [0.4660, 0.6740, 0.1880], 'MarkerSize',
8, ...
207      'DisplayName', '策略3: 保守控制');
208 plot([0, N-1], [0, 0], 'k--', 'LineWidth', 1.5, 'DisplayName', '零输入线');
209 xlabel('时间 t (时刻 k)', 'FontSize', 11, 'FontWeight', 'bold');
210 ylabel('控制输入 u_k', 'FontSize', 11, 'FontWeight', 'bold');
211 title('控制输入随时间变化', 'FontSize', 12, 'FontWeight', 'bold');
212 legend('Location', 'best');
213 xlim([0, N-1]);
214

```

```

215 % 系统状态随时间变化
216 subplot(1,3,2);
217 hold on; grid on; box on;
218 plot(t_x, x_opt, '-o', 'Color', [0, 0.4470, 0.7410], ...
219      'LineWidth', 2, 'MarkerFaceColor', [0, 0.4470, 0.7410], 'MarkerSize', 8,
220      'DisplayName', '策略1: LQR最优');
221 plot(t_x, x_strong, '-s', 'Color', [0.8500, 0.3250, 0.0980], ...
222      'LineWidth', 2, 'MarkerFaceColor', [0.8500, 0.3250, 0.0980], 'MarkerSize',
223      8, ...
224      'DisplayName', '策略2: 较强反馈');
225 plot(t_x, x_con, '-^', 'Color', [0.4660, 0.6740, 0.1880], ...
226      'LineWidth', 2, 'MarkerFaceColor', [0.4660, 0.6740, 0.1880], 'MarkerSize',
227      8, ...
228      'DisplayName', '策略3: 保守控制');
229 plot(t_x, x_open, 'm--', 'LineWidth', 1.5, 'DisplayName', '开环系统');
230 plot([0, N], [0, 0], 'k--', 'LineWidth', 1.5, 'DisplayName', '目标状态');
231 xlabel('时间 t (时刻 k)', 'FontSize', 11, 'FontWeight', 'bold');
232 ylabel('系统状态 x_k', 'FontSize', 11, 'FontWeight', 'bold');
233 title('系统状态随时间变化', 'FontSize', 12, 'FontWeight', 'bold');
234 legend('Location', 'best');
235 xlim([0, N]);
236
237 % 累计代价随时间变化
238 subplot(1,3,3);
239 hold on; grid on; box on;
240 plot(t_x, J_opt_cum, '-o', 'Color', [0, 0.4470, 0.7410], ...
241      'LineWidth', 2, 'MarkerFaceColor', [0, 0.4470, 0.7410], 'MarkerSize', 8,
242      ...
243      'DisplayName', '策略1: LQR最优');
244 plot(t_x, J_strong_cum, '-s', 'Color', [0.8500, 0.3250, 0.0980], ...
245      'LineWidth', 2, 'MarkerFaceColor', [0.8500, 0.3250, 0.0980], 'MarkerSize',
246      8, ...
247      'DisplayName', '策略2: 较强反馈');
248 plot(t_x, J_con_cum, '-^', 'Color', [0.4660, 0.6740, 0.1880], ...
249      'LineWidth', 2, 'MarkerFaceColor', [0.4660, 0.6740, 0.1880], 'MarkerSize',
250      8, ...
251      'DisplayName', '策略3: 保守控制');
252 xlabel('时间 t (时刻 k)', 'FontSize', 11, 'FontWeight', 'bold');
253 ylabel('累计代价 J_{cum}(k)', 'FontSize', 11, 'FontWeight', 'bold');
254 title('累计代价随时间变化', 'FontSize', 12, 'FontWeight', 'bold');
255 legend('Location', 'northwest');
256 xlim([0, N]);
257
258 % 标注最终总代价
259 text(N, J_opt_cum(end), sprintf('%.1f', J_opt_cum(end)), ...

```

```

255     'FontSize', 10, 'HorizontalAlignment', 'left', 'VerticalAlignment',
    'middle', 'Color', [0, 0.4470, 0.7410]);
256 text(N, J_strong_cum(end), sprintf('%.1f', J_strong_cum(end)), ...
257     'FontSize', 10, 'HorizontalAlignment', 'left', 'VerticalAlignment',
    'middle', 'Color', [0.8500, 0.3250, 0.0980]);
258 text(N, J_con_cum(end), sprintf('%.1f', J_con_cum(end)), ...
259     'FontSize', 10, 'HorizontalAlignment', 'left', 'VerticalAlignment',
    'middle', 'Color', [0.4660, 0.6740, 0.1880]);
260
261 sgtitle('离散系统时间域分析：控制输入、系统状态与累计代价', 'FontSize', 16,
    'FontWeight', 'bold');
262
263 %% 输出总结信息
264 fprintf('=== 三种控制策略的性能分析 ===\n');
265 fprintf('策略1 (LQR最优, K=%.4f): 总代价 = %.4f\n', K_opt, total_cost_opt);
266 fprintf('策略2 (较强反馈, K=%.4f): 总代价 = %.4f\n', K_strong,
    total_cost_strong);
267 fprintf('策略3 (保守控制, K=%.4f): 总代价 = %.4f\n', K_con, total_cost_con);
268 fprintf('\n最优性分析:\n');
269 fprintf('LQR最优策略的总代价最小, 比策略2低 %.2f%%\n', (total_cost_strong-
    total_cost_opt)/total_cost_strong*100);
270 fprintf('LQR最优策略的总代价最小, 比策略3低 %.2f%%\n', (total_cost_con-
    total_cost_opt)/total_cost_con*100);
271 fprintf('\n控制效果分析:\n');
272 fprintf('策略1 (LQR最优): 控制力度适中, 收敛平滑, 无振荡\n');
273 fprintf('策略2 (较强反馈): 控制力度较大, 收敛较快, 但控制能量消耗较多\n');
274 fprintf('策略3 (保守控制): 控制力度很小, 收敛缓慢, 状态误差持续存在时间长\n');

```

连续系统，累计价值从0向N计算的情况

代码块

```

1  %% 连续系统LQR控制策略对比分析
2  clear; clc; close all;
3
4  % 系统参数
5  A = 0.5;      % 系统矩阵
6  B = 1;        % 控制矩阵
7  x0 = 10;      % 初始状态
8  T = 8;        % 仿真时间 (秒)
9  Q = 1;        % 状态代价权重
10 R = 1;        % 控制代价权重
11
12 % 时间向量
13 dt = 0.01;    % 时间步长
14 t = 0:dt:T;   % 时间点

```

```

15 N = length(t); % 时间点数量
16
17 %% 1. 计算连续时间LQR最优增益
18 % 求解连续时间代数Riccati方程 (CARE)
19 P = care(A, B, Q, R); % 求解Riccati方程
20 K_opt = R\B'*P; % 最优反馈增益 K_opt ≈ 1.6180
21
22 % 策略定义
23 K_strong = 2; % 较强反馈增益 由于是一阶连续系统，怎么大都不会出现超调
24 K_con = 1.3; % 保守反馈增益
25
26 fprintf('=== 连续系统控制策略分析 ===\n');
27 fprintf('LQR最优增益: K_opt = %.4f\n', K_opt);
28 fprintf('较强反馈增益: K_strong = %.4f\n', K_strong);
29 fprintf('保守控制增益: K_con = %.4f\n\n', K_con);
30
31 %% 2. 策略1: LQR最优控制
32 % 闭环系统矩阵
33 A_cl_opt = A - B*K_opt;
34
35 % 求解状态方程
36 x_opt = zeros(1, N);
37 u_opt = zeros(1, N);
38 J_opt_single = zeros(1, N);
39 J_opt_cum = zeros(1, N);
40
41 x_opt(1) = x0;
42 u_opt(1) = -K_opt * x_opt(1);
43 J_opt_single(1) = (x_opt(1)^2 + u_opt(1)^2) * dt;
44 J_opt_cum(1) = J_opt_single(1);
45
46 % 数值积分求解
47 for k = 2:N
48     % 使用欧拉法 (也可用ode45, 但这里为简单用欧拉法)
49     x_opt(k) = x_opt(k-1) + (A_cl_opt * x_opt(k-1)) * dt;
50     u_opt(k) = -K_opt * x_opt(k);
51     J_opt_single(k) = (x_opt(k)^2 + u_opt(k)^2) * dt;
52     J_opt_cum(k) = J_opt_cum(k-1) + J_opt_single(k);
53 end
54
55 % 解析解 (用于验证)
56 % 连续系统解析解:  $x(t) = x_0 * \exp((A-B*K_{opt})*t)$ 
57 t_analytic = linspace(0, T, 1000);
58 x_opt_analytic = x0 * exp((A - B*K_opt) * t_analytic);
59 u_opt_analytic = -K_opt * x_opt_analytic;
60
61 %% 3. 策略2: 较强反馈控制

```

```

62  A_cl_strong = A - B*K_strong;
63
64  x_strong = zeros(1, N);
65  u_strong = zeros(1, N);
66  J_strong_single = zeros(1, N);
67  J_strong_cum = zeros(1, N);
68
69  x_strong(1) = x0;
70  u_strong(1) = -K_strong * x_strong(1);
71  J_strong_single(1) = (x_strong(1)^2 + u_strong(1)^2) * dt;
72  J_strong_cum(1) = J_strong_single(1);
73
74  for k = 2:N
75      x_strong(k) = x_strong(k-1) + (A_cl_strong * x_strong(k-1)) * dt;
76      u_strong(k) = -K_strong * x_strong(k);
77      J_strong_single(k) = (x_strong(k)^2 + u_strong(k)^2) * dt;
78      J_strong_cum(k) = J_strong_cum(k-1) + J_strong_single(k);
79  end
80
81  %% 4. 策略3: 保守控制
82  A_cl_con = A - B*K_con;
83
84  x_con = zeros(1, N);
85  u_con = zeros(1, N);
86  J_con_single = zeros(1, N);
87  J_con_cum = zeros(1, N);
88
89  x_con(1) = x0;
90  u_con(1) = -K_con * x_con(1);
91  J_con_single(1) = (x_con(1)^2 + u_con(1)^2) * dt;
92  J_con_cum(1) = J_con_single(1);
93
94  for k = 2:N
95      x_con(k) = x_con(k-1) + (A_cl_con * x_con(k-1)) * dt;
96      u_con(k) = -K_con * x_con(k);
97      J_con_single(k) = (x_con(k)^2 + u_con(k)^2) * dt;
98      J_con_cum(k) = J_con_cum(k-1) + J_con_single(k);
99  end
100
101  %% 5. 计算开环系统响应 (无控制)
102  % 开环系统:  $dx/dt = A*x$ , 无控制输入
103  x_open = x0 * exp(A * t); % 解析解
104  J_open_single = x_open.^2 * dt;
105  J_open_cum = cumsum(J_open_single);
106
107  %% 6. 计算最终总代价 (无穷时间代价的有限时间近似)
108  total_cost_opt = J_opt_cum(end);

```

```

109 total_cost_strong = J_strong_cum(end);
110 total_cost_con = J_con_cum(end);
111 total_cost_open = J_open_cum(end);
112
113 %% 7. 绘制控制输入u随时间变化，z轴为累计代价
114 figure('Color', 'w', 'Position', [100, 100, 1200, 900]);
115
116 % 创建三维坐标系
117 ax1 = subplot(2,2,[1,3]);
118 hold on; grid on; box on;
119 view(45, 30);
120
121 % 由于连续系统数据点太多，我们抽样绘制以减少绘图负担
122 skip = 20; % 每隔20个点绘制一个
123 t_sampled = t(1:skip:end);
124 u_opt_sampled = u_opt(1:skip:end);
125 u_strong_sampled = u_strong(1:skip:end);
126 u_con_sampled = u_con(1:skip:end);
127 J_opt_cum_sampled = J_opt_cum(1:skip:end);
128 J_strong_cum_sampled = J_strong_cum(1:skip:end);
129 J_con_cum_sampled = J_con_cum(1:skip:end);
130
131 % 绘制三条控制输入轨迹
132 % 策略1: LQR最优 (蓝色)
133 plot3(t_sampled, u_opt_sampled, J_opt_cum_sampled, '-', 'Color', [0, 0.4470,
0.7410], ...
134       'LineWidth', 2, 'DisplayName', sprintf('策略1: LQR最优 (K=%.3f, J=%.2f)',
K_opt, total_cost_opt));
135
136 % 策略2: 较强反馈 (红色)
137 plot3(t_sampled, u_strong_sampled, J_strong_cum_sampled, '-', 'Color',
[0.8500, 0.3250, 0.0980], ...
138       'LineWidth', 2, 'DisplayName', sprintf('策略2: 较强反馈 (K=%.3f, J=%.2f)',
K_strong, total_cost_strong));
139
140 % 策略3: 保守控制 (绿色)
141 plot3(t_sampled, u_con_sampled, J_con_cum_sampled, '-', 'Color', [0.4660,
0.6740, 0.1880], ...
142       'LineWidth', 2, 'DisplayName', sprintf('策略3: 保守控制 (K=%.3f, J=%.2f)',
K_con, total_cost_con));
143
144 % 在关键时间点添加标记
145 key_times = [0, 1, 2, 4, 8];
146 for kt = 1:length(key_times)
147     [~, idx] = min(abs(t - key_times(kt)));
148     if idx <= length(t)
149         % 策略1标记

```

```

150         plot3(t(idx), u_opt(idx), J_opt_cum(idx), 'o', ...
151             'MarkerFaceColor', [0, 0.4470, 0.7410], 'MarkerEdgeColor', 'k',
152             'MarkerSize', 8);
153         text(t(idx), u_opt(idx), J_opt_cum(idx)+5, ...
154             sprintf('t=%.1f\nu=%.2f', t(idx), u_opt(idx)), ...
155             'FontSize', 8, 'HorizontalAlignment', 'center', 'Color', [0,
156             0.4470, 0.7410]);
157
158         % 策略2标记
159         plot3(t(idx), u_strong(idx), J_strong_cum(idx), 's', ...
160             'MarkerFaceColor', [0.8500, 0.3250, 0.0980], 'MarkerEdgeColor',
161             'k', 'MarkerSize', 8);
162
163         % 策略3标记
164         plot3(t(idx), u_con(idx), J_con_cum(idx), '^', ...
165             'MarkerFaceColor', [0.4660, 0.6740, 0.1880], 'MarkerEdgeColor',
166             'k', 'MarkerSize', 8);
167     end
168 end
169
170 % 绘制零输入线
171 plot3([0, T], [0, 0], [0, 0], 'k--', 'LineWidth', 1.5, ...
172     'DisplayName', '零输入线 (u=0)');
173
174 % 添加坐标轴标签
175 xlabel('时间 t (秒)', 'FontSize', 12, 'FontWeight', 'bold');
176 ylabel('控制输入 u(t)', 'FontSize', 12, 'FontWeight', 'bold');
177 zlabel('累计代价 J_{cum}(t) = \int_0^t (x(\tau)^2 + u(\tau)^2) d\tau', 'FontSize', 12,
178     'FontWeight', 'bold');
179 title('控制输入u随时间变化, z轴为累计代价', 'FontSize', 14, 'FontWeight', 'bold');
180 legend('Location', 'best', 'FontSize', 10);
181 grid on;
182
183 % 设置坐标轴范围
184 xlim([0, T]);
185 u_max = max([abs(u_opt), abs(u_strong), abs(u_con)]);
186 ylim([-u_max*1.1, u_max*1.1]);
187 zlim([0, max([J_opt_cum, J_strong_cum, J_con_cum])*1.1]);
188
189 %% 8. 图2: 状态x随时间变化, z轴为累计代价
190 ax2 = subplot(2,2,[2,4]);
191 hold on; grid on; box on;
192 view(45, 30);
193
194 % 抽样数据
195 x_opt_sampled = x_opt(1:skip:end);
196 x_strong_sampled = x_strong(1:skip:end);

```

```

192 x_con_sampled = x_con(1:skip:end);
193
194 % 绘制三条状态轨迹
195 % 策略1: LQR最优 (蓝色)
196 plot3(t_sampled, x_opt_sampled, J_opt_cum_sampled, '-', 'Color', [0, 0.4470,
0.7410], ...
197     'LineWidth', 2, 'DisplayName', sprintf('策略1: LQR最优 (K=%.3f, J=%.2f)',
K_opt, total_cost_opt));
198
199 % 策略2: 较强反馈 (红色)
200 plot3(t_sampled, x_strong_sampled, J_strong_cum_sampled, '-', 'Color',
[0.8500, 0.3250, 0.0980], ...
201     'LineWidth', 2, 'DisplayName', sprintf('策略2: 较强反馈 (K=%.3f, J=%.2f)',
K_strong, total_cost_strong));
202
203 % 策略3: 保守控制 (绿色)
204 plot3(t_sampled, x_con_sampled, J_con_cum_sampled, '-', 'Color', [0.4660,
0.6740, 0.1880], ...
205     'LineWidth', 2, 'DisplayName', sprintf('策略3: 保守控制 (K=%.3f, J=%.2f)',
K_con, total_cost_con));
206
207 % 在关键时间点添加标记
208 for kt = 1:length(key_times)
209     [~, idx] = min(abs(t - key_times(kt)));
210     if idx <= length(t)
211         % 策略1标记
212         plot3(t(idx), x_opt(idx), J_opt_cum(idx), 'o', ...
213             'MarkerFaceColor', [0, 0.4470, 0.7410], 'MarkerEdgeColor', 'k',
'MarkerSize', 8);
214         text(t(idx), x_opt(idx), J_opt_cum(idx)+5, ...
215             sprintf('t=%.1f\nx=%.2f', t(idx), x_opt(idx)), ...
216             'FontSize', 8, 'HorizontalAlignment', 'center', 'Color', [0,
0.4470, 0.7410]);
217
218         % 策略2标记
219         plot3(t(idx), x_strong(idx), J_strong_cum(idx), 's', ...
220             'MarkerFaceColor', [0.8500, 0.3250, 0.0980], 'MarkerEdgeColor',
'k', 'MarkerSize', 8);
221
222         % 策略3标记
223         plot3(t(idx), x_con(idx), J_con_cum(idx), '^', ...
224             'MarkerFaceColor', [0.4660, 0.6740, 0.1880], 'MarkerEdgeColor',
'k', 'MarkerSize', 8);
225     end
226 end
227
228 % 绘制目标状态线

```



```

229 plot3([0, T], [0, 0], [0, 0], 'k--', 'LineWidth', 1.5, ...
230         'DisplayName', '目标状态 (x=0)');
231
232 % 绘制开环系统响应 (无控制)
233 x_open_sampled = x_open(1:skip:end);
234 plot3(t_sampled, x_open_sampled, zeros(size(t_sampled)), 'm--', 'LineWidth',
235         1.5, ...
236         'DisplayName', '开环系统 (无控制)');
237
238 % 添加坐标轴标签
239 xlabel('时间 t (秒)', 'FontSize', 12, 'FontWeight', 'bold');
240 ylabel('系统状态 x(t)', 'FontSize', 12, 'FontWeight', 'bold');
241 zlabel('累计代价 J_{cum}(t) = \int_0^t (x(\tau)^2 + u(\tau)^2) d\tau', 'FontSize', 12,
242         'FontWeight', 'bold');
243 title('系统状态x随时间变化, z轴为累计代价', 'FontSize', 14, 'FontWeight', 'bold');
244 legend('Location', 'best', 'FontSize', 10);
245 grid on;
246
247 % 设置坐标轴范围
248 xlim([0, T]);
249 x_max = max([abs(x_opt), abs(x_strong), abs(x_con), abs(x_open)]);
250 ylim([-x_max*0.1, x_max*1.1]);
251 zlim([0, max([J_opt_cum, J_strong_cum, J_con_cum])*1.1]);
252
253 %% 9. 绘制二维投影对比图
254 figure('Color', 'w', 'Position', [200, 200, 1200, 500]);
255
256 % 控制输入随时间变化
257 subplot(1,3,1);
258 hold on; grid on; box on;
259 plot(t, u_opt, '-', 'Color', [0, 0.4470, 0.7410], 'LineWidth', 2,
260         'DisplayName', '策略1: LQR最优');
261 plot(t, u_strong, '-', 'Color', [0.8500, 0.3250, 0.0980], 'LineWidth', 2,
262         'DisplayName', '策略2: 较强反馈');
263 plot(t, u_con, '-', 'Color', [0.4660, 0.6740, 0.1880], 'LineWidth', 2,
264         'DisplayName', '策略3: 保守控制');
265 plot([0, T], [0, 0], 'k--', 'LineWidth', 1.5, 'DisplayName', '零输入线');
266 xlabel('时间 t (秒)', 'FontSize', 11, 'FontWeight', 'bold');
267 ylabel('控制输入 u(t)', 'FontSize', 11, 'FontWeight', 'bold');
268 title('控制输入随时间变化', 'FontSize', 12, 'FontWeight', 'bold');
269 legend('Location', 'best');
270 xlim([0, T]);
271
272 % 系统状态随时间变化
273 subplot(1,3,2);
274 hold on; grid on; box on;

```

```

270 plot(t, x_opt, '-', 'Color', [0, 0.4470, 0.7410], 'LineWidth', 2,
      'DisplayName', '策略1: LQR最优');
271 plot(t, x_strong, '-', 'Color', [0.8500, 0.3250, 0.0980], 'LineWidth', 2,
      'DisplayName', '策略2: 较强反馈');
272 plot(t, x_con, '-', 'Color', [0.4660, 0.6740, 0.1880], 'LineWidth', 2,
      'DisplayName', '策略3: 保守控制');
273 plot(t, x_open, 'm--', 'LineWidth', 1.5, 'DisplayName', '开环系统');
274 plot([0, T], [0, 0], 'k--', 'LineWidth', 1.5, 'DisplayName', '目标状态');
275 xlabel('时间 t (秒)', 'FontSize', 11, 'FontWeight', 'bold');
276 ylabel('系统状态 x(t)', 'FontSize', 11, 'FontWeight', 'bold');
277 title('系统状态随时间变化', 'FontSize', 12, 'FontWeight', 'bold');
278 legend('Location', 'best');
279 xlim([0, T]);
280
281 % 累计代价随时间变化
282 subplot(1,3,3);
283 hold on; grid on; box on;
284 plot(t, J_opt_cum, '-', 'Color', [0, 0.4470, 0.7410], 'LineWidth', 2,
      'DisplayName', '策略1: LQR最优');
285 plot(t, J_strong_cum, '-', 'Color', [0.8500, 0.3250, 0.0980], 'LineWidth', 2,
      'DisplayName', '策略2: 较强反馈');
286 plot(t, J_con_cum, '-', 'Color', [0.4660, 0.6740, 0.1880], 'LineWidth', 2,
      'DisplayName', '策略3: 保守控制');
287 plot(t, J_open_cum, 'm--', 'LineWidth', 1.5, 'DisplayName', '开环系统');
288 xlabel('时间 t (秒)', 'FontSize', 11, 'FontWeight', 'bold');
289 ylabel('累计代价 J_{cum}(t)', 'FontSize', 11, 'FontWeight', 'bold');
290 title('累计代价随时间变化', 'FontSize', 12, 'FontWeight', 'bold');
291 legend('Location', 'northwest');
292 xlim([0, T]);
293
294 % 标注最终总代价
295 text(T, J_opt_cum(end), sprintf('%.1f', J_opt_cum(end)), ...
      'FontSize', 10, 'HorizontalAlignment', 'left', 'VerticalAlignment',
      'middle', 'Color', [0, 0.4470, 0.7410]);
296 text(T, J_strong_cum(end), sprintf('%.1f', J_strong_cum(end)), ...
      'FontSize', 10, 'HorizontalAlignment', 'left', 'VerticalAlignment',
      'middle', 'Color', [0.8500, 0.3250, 0.0980]);
297 text(T, J_con_cum(end), sprintf('%.1f', J_con_cum(end)), ...
      'FontSize', 10, 'HorizontalAlignment', 'left', 'VerticalAlignment',
      'middle', 'Color', [0.4660, 0.6740, 0.1880]);
298 text(T, J_open_cum(end), sprintf('%.1f', J_open_cum(end)), ...
      'FontSize', 10, 'HorizontalAlignment', 'left', 'VerticalAlignment',
      'middle', 'Color', 'm');
299
300 sgtitle('连续系统时间域分析: 控制输入、系统状态与累计代价', 'FontSize', 16,
      'FontWeight', 'bold');
301
302
303
304
305

```

```

306 %% 10. 绘制闭环系统极点位置
307 figure('Color', 'w', 'Position', [300, 300, 800, 600]);
308 hold on; grid on; box on;
309
310 % 计算闭环系统极点
311 p_opt = A - B*K_opt;
312 p_strong = A - B*K_strong;
313 p_con = A - B*K_con;
314 p_open = A; % 开环系统极点
315
316 % 绘制极点
317 plot(real(p_opt), imag(p_opt), 'o', 'MarkerSize', 12, 'MarkerFaceColor', [0,
0.4470, 0.7410], ...
318      'MarkerEdgeColor', 'k', 'DisplayName', sprintf('策略1: LQR最优 ( $\lambda=0.3f$ )',
p_opt));
319 plot(real(p_strong), imag(p_strong), 's', 'MarkerSize', 12, 'MarkerFaceColor',
[0.8500, 0.3250, 0.0980], ...
320      'MarkerEdgeColor', 'k', 'DisplayName', sprintf('策略2: 较强反馈 ( $\lambda=0.3f$ )',
p_strong));
321 plot(real(p_con), imag(p_con), '^', 'MarkerSize', 12, 'MarkerFaceColor',
[0.4660, 0.6740, 0.1880], ...
322      'MarkerEdgeColor', 'k', 'DisplayName', sprintf('策略3: 保守控制 ( $\lambda=0.3f$ )',
p_con));
323 plot(real(p_open), imag(p_open), 'd', 'MarkerSize', 12, 'MarkerFaceColor',
'm', ...
324      'MarkerEdgeColor', 'k', 'DisplayName', sprintf('开环系统 ( $\lambda=0.3f$ )',
p_open));
325
326 % 绘制稳定区域边界 (左半平面)
327 plot([-5, 0], [0, 0], 'r--', 'LineWidth', 1.5, 'DisplayName', '稳定边界
(Re=0)');
328 plot([0, 0], [-2, 2], 'r--', 'LineWidth', 1.5);
329
330 % 设置坐标轴
331 xlim([-2, 0.6]);
332 ylim([-0.5, 0.5]);
333 xlabel('实部 (Re)', 'FontSize', 12, 'FontWeight', 'bold');
334 ylabel('虚部 (Im)', 'FontSize', 12, 'FontWeight', 'bold');
335 title('闭环系统极点位置', 'FontSize', 14, 'FontWeight', 'bold');
336 legend('Location', 'best');
337
338 % 填充稳定区域
339 fill([-5, 0, 0, -5], [-2, -2, 2, 2], [1, 0.9, 0.9], 'FaceAlpha', 0.3,
'EdgeColor', 'none');
340 text(-1, 0.3, '稳定区域', 'FontSize', 12, 'Color', 'r', 'FontWeight', 'bold');
341
342 %% 11. 输出总结信息

```

```

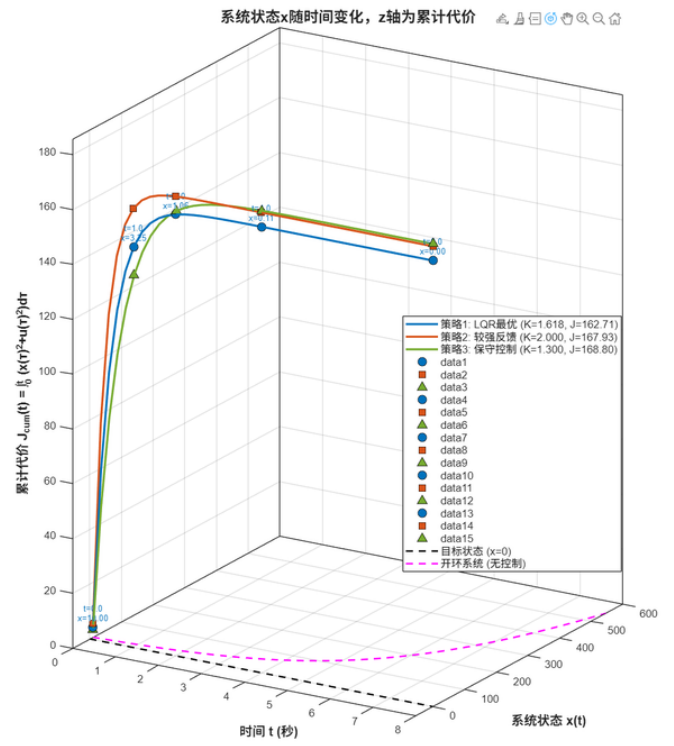
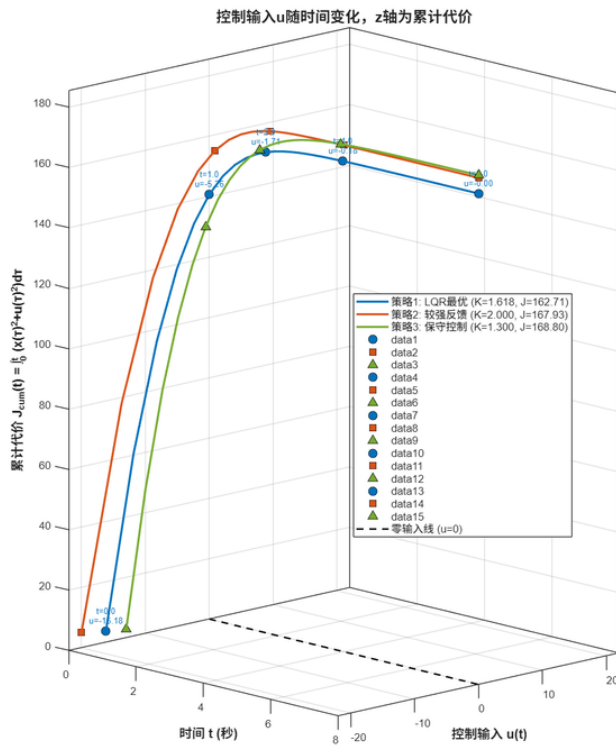
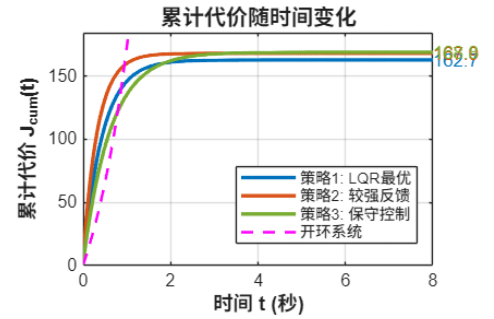
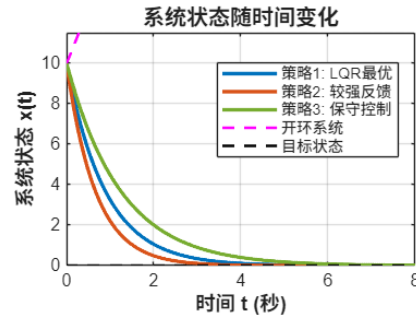
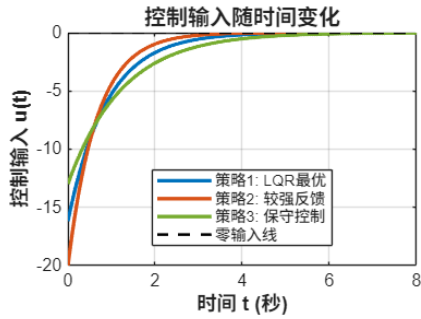
343 fprintf('=== 连续系统控制策略性能分析 ===\n');
344 fprintf('策略1 (LQR最优, K=%.4f): 总代价 = %.4f\n', K_opt, total_cost_opt);
345 fprintf('策略2 (较强反馈, K=%.4f): 总代价 = %.4f\n', K_strong,
total_cost_strong);
346 fprintf('策略3 (保守控制, K=%.4f): 总代价 = %.4f\n', K_con, total_cost_con);
347 fprintf('开环系统 (无控制): 总代价 = %.4f\n\n', total_cost_open);
348
349 fprintf('闭环系统极点分析:\n');
350 fprintf('策略1 (LQR最优): 闭环极点  $\lambda$  = %.4f, 收敛速率 = %.4f\n', p_opt, -p_opt);
351 fprintf('策略2 (较强反馈): 闭环极点  $\lambda$  = %.4f, 收敛速率 = %.4f\n', p_strong, -
p_strong);
352 fprintf('策略3 (保守控制): 闭环极点  $\lambda$  = %.4f, 收敛速率 = %.4f\n', p_con, -p_con);
353 fprintf('开环系统: 开环极点  $\lambda$  = %.4f, 收敛速率 = %.4f\n\n', p_open, -p_open);
354
355 fprintf('最优性分析:\n');
356 fprintf('LQR最优策略的总代价最小, 比策略2低 %.2f%%\n', (total_cost_strong-
total_cost_opt)/total_cost_strong*100);
357 fprintf('LQR最优策略的总代价最小, 比策略3低 %.2f%%\n', (total_cost_con-
total_cost_opt)/total_cost_con*100);
358 fprintf('LQR最优策略的总代价最小, 比开环系统低 %.2f%%\n\n', (total_cost_open-
total_cost_opt)/total_cost_open*100);
359
360 fprintf('控制效果分析:\n');
361 fprintf('策略1 (LQR最优): 控制力度适中, 收敛平滑, 代价最小\n');
362 fprintf('策略2 (较强反馈): 控制力度较大, 收敛较快, 但控制能量消耗较多\n');
363 fprintf('策略3 (保守控制): 控制力度很小, 收敛缓慢, 状态误差持续存在时间长\n');
364 fprintf('开环系统: 无控制, 收敛最慢, 代价最大\n');
365
366 %% 12. 验证解析解与数值解的一致性
367 figure('Color', 'w', 'Position', [400, 200, 800, 400]);
368 subplot(1,2,1);
369 hold on; grid on; box on;
370 plot(t_analytic, x_opt_analytic, 'b-', 'LineWidth', 2, 'DisplayName', '解析解');
371 plot(t, x_opt, 'r--', 'LineWidth', 1.5, 'DisplayName', '数值解');
372 xlabel('时间 t (秒)', 'FontSize', 11, 'FontWeight', 'bold');
373 ylabel('系统状态 x(t)', 'FontSize', 11, 'FontWeight', 'bold');
374 title('LQR最优控制: 解析解与数值解对比', 'FontSize', 12, 'FontWeight', 'bold');
375 legend('Location', 'best');
376
377 % 计算相对误差
378 rel_error = abs(x_opt(1:10:end) - interp1(t_analytic, x_opt_analytic,
t(1:10:end))) ./ ...
379     abs(interp1(t_analytic, x_opt_analytic, t(1:10:end)));
380 max_rel_error = max(rel_error);
381
382 subplot(1,2,2);
383 plot(t(1:10:end), rel_error, 'k-', 'LineWidth', 1.5);

```

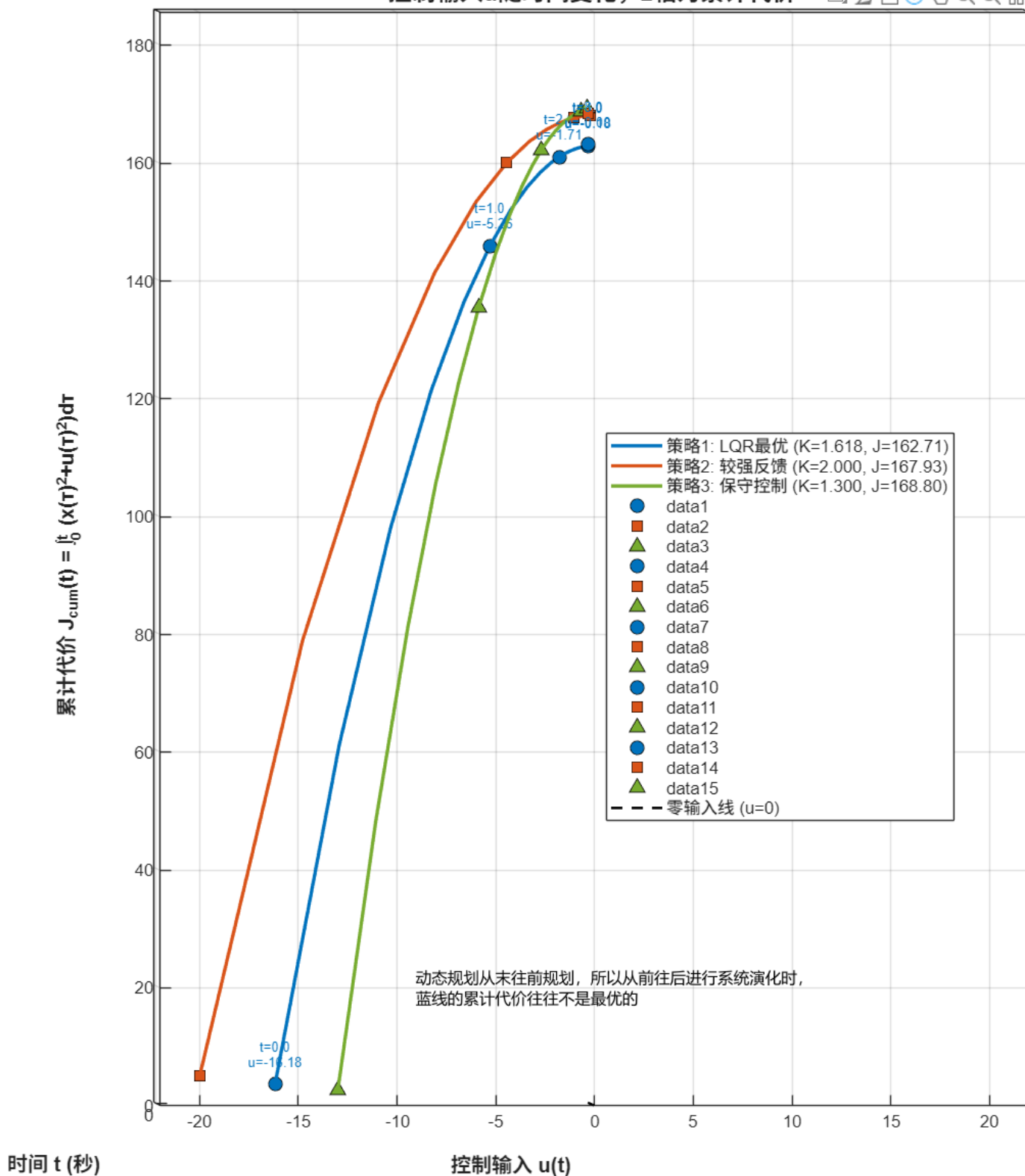
```

384 grid on; box on;
385 xlabel('时间 t (秒)', 'FontSize', 11, 'FontWeight', 'bold');
386 ylabel('相对误差', 'FontSize', 11, 'FontWeight', 'bold');
387 title(sprintf('相对误差 (最大: %.2e)', max_rel_error), 'FontSize', 12,
    'FontWeight', 'bold');

```



控制输入u随时间变化，z轴为累计代价



从初始状态开始累加，加出来的就是这种开始不是最优，到一定步之后才达到最优的效果。真实系统是从前往后，假想的优化系统是从后往前。

另外 这种Jcum的算法其实有问题，其实每一步应该都是Jcum = 从x0加到xk+1(因为优化还涉及到额外的一步)；或者直接从x0加到xN,而不是当前步，这是因为泛函对u求偏导的时候是对整个从x0到xN求的偏导。求偏导为0的应该在上一节展示了。

当然，这种以加到当前步为止的Jcum算法也可以作为一个物理量进行参考。

以下是一个其他策略不稳定的例子，用这个例子说明LQR解一定是满足李雅普诺夫稳定的

代码块

```
1 %% 连续系统LQR控制策略对比分析
2 clear; clc; close all;
3
4 % 系统参数
5 A = 0.5; % 系统矩阵
6 B = 1; % 控制矩阵
7 x0 = 10; % 初始状态
8 T = 8; % 仿真时间 (秒)
9 Q = 1; % 状态代价权重
10 R = 1; % 控制代价权重
11
12 % 时间向量
13 dt = 0.01; % 时间步长
14 t = 0:dt:T; % 时间点
15 N = length(t); % 时间点数量
16
17 %% 1. 计算连续时间LQR最优增益
18 % 求解连续时间代数Riccati方程 (CARE)
19 P = care(A, B, Q, R); % 求解Riccati方程
20 K_opt = R\B'*P; % 最优反馈增益 K_opt ≈ 0.6180
21
22 % 策略定义
23 K_strong = 1.2; % 较强反馈增益
24 K_con = 0.1; % 保守反馈增益
25
26 fprintf('=== 连续系统控制策略分析 ===\n');
27 fprintf('LQR最优增益: K_opt = %.4f\n', K_opt);
28 fprintf('较强反馈增益: K_strong = %.4f\n', K_strong);
29 fprintf('保守控制增益: K_con = %.4f\n\n', K_con);
30
31 %% 2. 策略1: LQR最优控制
32 % 闭环系统矩阵
33 A_cl_opt = A - B*K_opt;
34
35 % 求解状态方程
36 x_opt = zeros(1, N);
37 u_opt = zeros(1, N);
38 J_opt_single = zeros(1, N);
39 J_opt_cum = zeros(1, N);
40
41 x_opt(1) = x0;
42 u_opt(1) = -K_opt * x_opt(1);
```

```

43 J_opt_single(1) = (x_opt(1)^2 + u_opt(1)^2) * dt;
44 J_opt_cum(1) = J_opt_single(1);
45
46 % 数值积分求解
47 for k = 2:N
48     % 使用欧拉法 (也可用ode45, 但这里为简单用欧拉法)
49     x_opt(k) = x_opt(k-1) + (A_cl_opt * x_opt(k-1)) * dt;
50     u_opt(k) = -K_opt * x_opt(k);
51     J_opt_single(k) = (x_opt(k)^2 + u_opt(k)^2) * dt;
52     J_opt_cum(k) = J_opt_cum(k-1) + J_opt_single(k);
53 end
54
55 % 解析解 (用于验证)
56 % 连续系统解析解:  $x(t) = x_0 * \exp((A-B*K_{opt})*t)$ 
57 t_analytic = linspace(0, T, 1000);
58 x_opt_analytic = x0 * exp((A - B*K_opt) * t_analytic);
59 u_opt_analytic = -K_opt * x_opt_analytic;
60
61 %% 3. 策略2: 较强反馈控制
62 A_cl_strong = A - B*K_strong;
63
64 x_strong = zeros(1, N);
65 u_strong = zeros(1, N);
66 J_strong_single = zeros(1, N);
67 J_strong_cum = zeros(1, N);
68
69 x_strong(1) = x0;
70 u_strong(1) = -K_strong * x_strong(1);
71 J_strong_single(1) = (x_strong(1)^2 + u_strong(1)^2) * dt;
72 J_strong_cum(1) = J_strong_single(1);
73
74 for k = 2:N
75     x_strong(k) = x_strong(k-1) + (A_cl_strong * x_strong(k-1)) * dt;
76     u_strong(k) = -K_strong * x_strong(k);
77     J_strong_single(k) = (x_strong(k)^2 + u_strong(k)^2) * dt;
78     J_strong_cum(k) = J_strong_cum(k-1) + J_strong_single(k);
79 end
80
81 %% 4. 策略3: 保守控制
82 A_cl_con = A - B*K_con;
83
84 x_con = zeros(1, N);
85 u_con = zeros(1, N);
86 J_con_single = zeros(1, N);
87 J_con_cum = zeros(1, N);
88
89 x_con(1) = x0;

```



```

90 u_con(1) = -K_con * x_con(1);
91 J_con_single(1) = (x_con(1)^2 + u_con(1)^2) * dt;
92 J_con_cum(1) = J_con_single(1);
93
94 for k = 2:N
95     x_con(k) = x_con(k-1) + (A_cl_con * x_con(k-1)) * dt;
96     u_con(k) = -K_con * x_con(k);
97     J_con_single(k) = (x_con(k)^2 + u_con(k)^2) * dt;
98     J_con_cum(k) = J_con_cum(k-1) + J_con_single(k);
99 end
100
101 %% 5. 计算开环系统响应 (无控制)
102 % 开环系统:  $dx/dt = A*x$ , 无控制输入
103 x_open = x0 * exp(A * t); % 解析解
104 J_open_single = x_open.^2 * dt;
105 J_open_cum = cumsum(J_open_single);
106
107 %% 6. 计算最终总代价 (无穷时间代价的有限时间近似)
108 total_cost_opt = J_opt_cum(end);
109 total_cost_strong = J_strong_cum(end);
110 total_cost_con = J_con_cum(end);
111 total_cost_open = J_open_cum(end);
112
113 %% 7. 绘制控制输入u随时间变化, z轴为累计代价
114 figure('Color', 'w', 'Position', [100, 100, 1200, 900]);
115
116 % 创建三维坐标系
117 ax1 = subplot(2,2,[1,3]);
118 hold on; grid on; box on;
119 view(45, 30);
120
121 % 由于连续系统数据点太多, 我们抽样绘制以减少绘图负担
122 skip = 20; % 每隔20个点绘制一个
123 t_sampled = t(1:skip:end);
124 u_opt_sampled = u_opt(1:skip:end);
125 u_strong_sampled = u_strong(1:skip:end);
126 u_con_sampled = u_con(1:skip:end);
127 J_opt_cum_sampled = J_opt_cum(1:skip:end);
128 J_strong_cum_sampled = J_strong_cum(1:skip:end);
129 J_con_cum_sampled = J_con_cum(1:skip:end);
130
131 % 绘制三条控制输入轨迹
132 % 策略1: LQR最优 (蓝色)
133 plot3(t_sampled, u_opt_sampled, J_opt_cum_sampled, '-','Color',[0, 0.4470, 0.7410], ...
134     'LineWidth', 2, 'DisplayName', sprintf('策略1: LQR最优 (K=%.3f, J=%.2f)',
135     K_opt, total_cost_opt));

```

```

135
136 % 策略2: 较强反馈 (红色)
137 plot3(t_sampled, u_strong_sampled, J_strong_cum_sampled, '-', 'Color',
138 [0.8500, 0.3250, 0.0980], ...
139 'LineWidth', 2, 'DisplayName', sprintf('策略2: 较强反馈 (K=%.3f, J=%.2f)',
140 K_strong, total_cost_strong));
141
142 % 策略3: 保守控制 (绿色)
143 plot3(t_sampled, u_con_sampled, J_con_cum_sampled, '-', 'Color', [0.4660,
144 0.6740, 0.1880], ...
145 'LineWidth', 2, 'DisplayName', sprintf('策略3: 保守控制 (K=%.3f, J=%.2f)',
146 K_con, total_cost_con));
147
148 % 在关键时间点添加标记
149 key_times = [0, 1, 2, 4, 8];
150 for kt = 1:length(key_times)
151     [~, idx] = min(abs(t - key_times(kt)));
152     if idx <= length(t)
153         % 策略1标记
154         plot3(t(idx), u_opt(idx), J_opt_cum(idx), 'o', ...
155             'MarkerFaceColor', [0, 0.4470, 0.7410], 'MarkerEdgeColor', 'k',
156             'MarkerSize', 8);
157         text(t(idx), u_opt(idx), J_opt_cum(idx)+5, ...
158             sprintf('t=%.1f\nu=%.2f', t(idx), u_opt(idx)), ...
159             'FontSize', 8, 'HorizontalAlignment', 'center', 'Color', [0,
160 0.4470, 0.7410]);
161
162         % 策略2标记
163         plot3(t(idx), u_strong(idx), J_strong_cum(idx), 's', ...
164             'MarkerFaceColor', [0.8500, 0.3250, 0.0980], 'MarkerEdgeColor',
165             'k', 'MarkerSize', 8);
166
167         % 策略3标记
168         plot3(t(idx), u_con(idx), J_con_cum(idx), '^', ...
169             'MarkerFaceColor', [0.4660, 0.6740, 0.1880], 'MarkerEdgeColor',
170             'k', 'MarkerSize', 8);
171     end
172 end
173
174 % 绘制零输入线
175 plot3([0, T], [0, 0], [0, 0], 'k--', 'LineWidth', 1.5, ...
176     'DisplayName', '零输入线 (u=0)');
177
178 % 添加坐标轴标签
179 xlabel('时间 t (秒)', 'FontSize', 12, 'FontWeight', 'bold');
180 ylabel('控制输入 u(t)', 'FontSize', 12, 'FontWeight', 'bold');

```

```

173 zlabel('累计代价 J_{cum}(t) = \int_0^t (x(\tau)^2+u(\tau)^2)d\tau', 'FontSize', 12,
174 'FontWeight', 'bold');
175 title('控制输入u随时间变化, z轴为累计代价', 'FontSize', 14, 'FontWeight', 'bold');
176 legend('Location', 'best', 'FontSize', 10);
177 grid on;
178 % 设置坐标轴范围
179 xlim([0, T]);
180 u_max = max([abs(u_opt), abs(u_strong), abs(u_con)]);
181 ylim([-u_max*1.1, u_max*1.1]);
182 zlim([0, max([J_opt_cum, J_strong_cum, J_con_cum])*1.1]);
183
184 %% 8. 图2: 状态x随时间变化, z轴为累计代价
185 ax2 = subplot(2,2,[2,4]);
186 hold on; grid on; box on;
187 view(45, 30);
188
189 % 抽样数据
190 x_opt_sampled = x_opt(1:skip:end);
191 x_strong_sampled = x_strong(1:skip:end);
192 x_con_sampled = x_con(1:skip:end);
193
194 % 绘制三条状态轨迹
195 % 策略1: LQR最优 (蓝色)
196 plot3(t_sampled, x_opt_sampled, J_opt_cum_sampled, '-', 'Color', [0, 0.4470,
197 0.7410], ...
198 'LineWidth', 2, 'DisplayName', sprintf('策略1: LQR最优 (K=%.3f, J=%.2f)',
199 K_opt, total_cost_opt));
200
201 % 策略2: 较强反馈 (红色)
202 plot3(t_sampled, x_strong_sampled, J_strong_cum_sampled, '-', 'Color',
203 [0.8500, 0.3250, 0.0980], ...
204 'LineWidth', 2, 'DisplayName', sprintf('策略2: 较强反馈 (K=%.3f, J=%.2f)',
205 K_strong, total_cost_strong));
206
207 % 策略3: 保守控制 (绿色)
208 plot3(t_sampled, x_con_sampled, J_con_cum_sampled, '-', 'Color', [0.4660,
209 0.6740, 0.1880], ...
210 'LineWidth', 2, 'DisplayName', sprintf('策略3: 保守控制 (K=%.3f, J=%.2f)',
211 K_con, total_cost_con));
212
213 % 在关键时间点添加标记
214 for kt = 1:length(key_times)
215     [~, idx] = min(abs(t - key_times(kt)));
216     if idx <= length(t)
217         % 策略1标记
218         plot3(t(idx), x_opt(idx), J_opt_cum(idx), 'o', ...

```

```

213         'MarkerFaceColor', [0, 0.4470, 0.7410], 'MarkerEdgeColor', 'k',
'MarkerSize', 8);
214         text(t(idx), x_opt(idx), J_opt_cum(idx)+5, ...
215             sprintf('t=%.1f\nx=%.2f', t(idx), x_opt(idx)), ...
216             'FontSize', 8, 'HorizontalAlignment', 'center', 'Color', [0,
0.4470, 0.7410]);
217
218         % 策略2标记
219         plot3(t(idx), x_strong(idx), J_strong_cum(idx), 's', ...
220             'MarkerFaceColor', [0.8500, 0.3250, 0.0980], 'MarkerEdgeColor',
'k', 'MarkerSize', 8);
221
222         % 策略3标记
223         plot3(t(idx), x_con(idx), J_con_cum(idx), '^', ...
224             'MarkerFaceColor', [0.4660, 0.6740, 0.1880], 'MarkerEdgeColor',
'k', 'MarkerSize', 8);
225     end
226 end
227
228 % 绘制目标状态线
229 plot3([0, T], [0, 0], [0, 0], 'k--', 'LineWidth', 1.5, ...
230     'DisplayName', '目标状态 (x=0)');
231
232 % 绘制开环系统响应 (无控制)
233 x_open_sampled = x_open(1:skip:end);
234 plot3(t_sampled, x_open_sampled, zeros(size(t_sampled)), 'm--', 'LineWidth',
1.5, ...
235     'DisplayName', '开环系统 (无控制)');
236
237 % 添加坐标轴标签
238 xlabel('时间 t (秒)', 'FontSize', 12, 'FontWeight', 'bold');
239 ylabel('系统状态 x(t)', 'FontSize', 12, 'FontWeight', 'bold');
240 zlabel('累计代价 J_{cum}(t) = \int_0^t (x(\tau)^2+u(\tau)^2)d\tau', 'FontSize', 12,
'FontWeight', 'bold');
241 title('系统状态x随时间变化, z轴为累计代价', 'FontSize', 14, 'FontWeight', 'bold');
242 legend('Location', 'best', 'FontSize', 10);
243 grid on;
244
245 % 设置坐标轴范围
246 xlim([0, T]);
247 x_max = max([abs(x_opt), abs(x_strong), abs(x_con), abs(x_open)]);
248 ylim([-x_max*0.1, x_max*1.1]);
249 zlim([0, max([J_opt_cum, J_strong_cum, J_con_cum])*1.1]);
250
251 %% 9. 绘制二维投影对比图
252 figure('Color', 'w', 'Position', [200, 200, 1200, 500]);
253

```

```

254 % 控制输入随时间变化
255 subplot(1,3,1);
256 hold on; grid on; box on;
257 plot(t, u_opt, '-', 'Color', [0, 0.4470, 0.7410], 'LineWidth', 2,
      'DisplayName', '策略1: LQR最优');
258 plot(t, u_strong, '-', 'Color', [0.8500, 0.3250, 0.0980], 'LineWidth', 2,
      'DisplayName', '策略2: 较强反馈');
259 plot(t, u_con, '-', 'Color', [0.4660, 0.6740, 0.1880], 'LineWidth', 2,
      'DisplayName', '策略3: 保守控制');
260 plot([0, T], [0, 0], 'k--', 'LineWidth', 1.5, 'DisplayName', '零输入线');
261 xlabel('时间 t (秒)', 'FontSize', 11, 'FontWeight', 'bold');
262 ylabel('控制输入 u(t)', 'FontSize', 11, 'FontWeight', 'bold');
263 title('控制输入随时间变化', 'FontSize', 12, 'FontWeight', 'bold');
264 legend('Location', 'best');
265 xlim([0, T]);
266
267 % 系统状态随时间变化
268 subplot(1,3,2);
269 hold on; grid on; box on;
270 plot(t, x_opt, '-', 'Color', [0, 0.4470, 0.7410], 'LineWidth', 2,
      'DisplayName', '策略1: LQR最优');
271 plot(t, x_strong, '-', 'Color', [0.8500, 0.3250, 0.0980], 'LineWidth', 2,
      'DisplayName', '策略2: 较强反馈');
272 plot(t, x_con, '-', 'Color', [0.4660, 0.6740, 0.1880], 'LineWidth', 2,
      'DisplayName', '策略3: 保守控制');
273 plot(t, x_open, 'm--', 'LineWidth', 1.5, 'DisplayName', '开环系统');
274 plot([0, T], [0, 0], 'k--', 'LineWidth', 1.5, 'DisplayName', '目标状态');
275 xlabel('时间 t (秒)', 'FontSize', 11, 'FontWeight', 'bold');
276 ylabel('系统状态 x(t)', 'FontSize', 11, 'FontWeight', 'bold');
277 title('系统状态随时间变化', 'FontSize', 12, 'FontWeight', 'bold');
278 legend('Location', 'best');
279 xlim([0, T]);
280
281 % 累计代价随时间变化
282 subplot(1,3,3);
283 hold on; grid on; box on;
284 plot(t, J_opt_cum, '-', 'Color', [0, 0.4470, 0.7410], 'LineWidth', 2,
      'DisplayName', '策略1: LQR最优');
285 plot(t, J_strong_cum, '-', 'Color', [0.8500, 0.3250, 0.0980], 'LineWidth', 2,
      'DisplayName', '策略2: 较强反馈');
286 plot(t, J_con_cum, '-', 'Color', [0.4660, 0.6740, 0.1880], 'LineWidth', 2,
      'DisplayName', '策略3: 保守控制');
287 plot(t, J_open_cum, 'm--', 'LineWidth', 1.5, 'DisplayName', '开环系统');
288 xlabel('时间 t (秒)', 'FontSize', 11, 'FontWeight', 'bold');
289 ylabel('累计代价 J_{cum}(t)', 'FontSize', 11, 'FontWeight', 'bold');
290 title('累计代价随时间变化', 'FontSize', 12, 'FontWeight', 'bold');
291 legend('Location', 'northwest');

```

```

292 xlim([0, T]);
293
294 % 标注最终总代价
295 text(T, J_opt_cum(end), sprintf('%1f', J_opt_cum(end)), ...
296     'FontSize', 10, 'HorizontalAlignment', 'left', 'VerticalAlignment',
297     'middle', 'Color', [0, 0.4470, 0.7410]);
298 text(T, J_strong_cum(end), sprintf('%1f', J_strong_cum(end)), ...
299     'FontSize', 10, 'HorizontalAlignment', 'left', 'VerticalAlignment',
300     'middle', 'Color', [0.8500, 0.3250, 0.0980]);
301 text(T, J_con_cum(end), sprintf('%1f', J_con_cum(end)), ...
302     'FontSize', 10, 'HorizontalAlignment', 'left', 'VerticalAlignment',
303     'middle', 'Color', 'm');
304
305
306 sgtitle('连续系统时间域分析：控制输入、系统状态与累计代价', 'FontSize', 16,
307     'FontWeight', 'bold');
308
309
310 %% 10. 绘制闭环系统极点位置
311 figure('Color', 'w', 'Position', [300, 300, 800, 600]);
312 hold on; grid on; box on;
313
314 % 计算闭环系统极点
315 p_opt = A - B*K_opt;
316 p_strong = A - B*K_strong;
317 p_con = A - B*K_con;
318 p_open = A; % 开环系统极点
319
320 % 绘制极点
321 plot(real(p_opt), imag(p_opt), 'o', 'MarkerSize', 12, 'MarkerFaceColor', [0,
322     0.4470, 0.7410], ...
323     'MarkerEdgeColor', 'k', 'DisplayName', sprintf('策略1: LQR最优 ( $\lambda=0.3f$ )',
324     p_opt));
325 plot(real(p_strong), imag(p_strong), 's', 'MarkerSize', 12, 'MarkerFaceColor',
326     [0.8500, 0.3250, 0.0980], ...
327     'MarkerEdgeColor', 'k', 'DisplayName', sprintf('策略2: 较强反馈 ( $\lambda=0.3f$ )',
328     p_strong));
329 plot(real(p_con), imag(p_con), '^', 'MarkerSize', 12, 'MarkerFaceColor',
330     [0.4660, 0.6740, 0.1880], ...
331     'MarkerEdgeColor', 'k', 'DisplayName', sprintf('策略3: 保守控制 ( $\lambda=0.3f$ )',
332     p_con));
333 plot(real(p_open), imag(p_open), 'd', 'MarkerSize', 12, 'MarkerFaceColor',
334     'm', ...
335     'MarkerEdgeColor', 'k', 'DisplayName', sprintf('开环系统 ( $\lambda=0.3f$ )',
336     p_open));
337
338

```

```

326 % 绘制稳定区域边界 (左半平面)
327 plot([-5, 0], [0, 0], 'r--', 'LineWidth', 1.5, 'DisplayName', '稳定边界
    (Re=0)');
328 plot([0, 0], [-2, 2], 'r--', 'LineWidth', 1.5);
329
330 % 设置坐标轴
331 xlim([-2, 0.6]);
332 ylim([-0.5, 0.5]);
333 xlabel('实部 (Re)', 'FontSize', 12, 'FontWeight', 'bold');
334 ylabel('虚部 (Im)', 'FontSize', 12, 'FontWeight', 'bold');
335 title('闭环系统极点位置', 'FontSize', 14, 'FontWeight', 'bold');
336 legend('Location', 'best');
337
338 % 填充稳定区域
339 fill([-5, 0, 0, -5], [-2, -2, 2, 2], [1, 0.9, 0.9], 'FaceAlpha', 0.3,
    'EdgeColor', 'none');
340 text(-1, 0.3, '稳定区域', 'FontSize', 12, 'Color', 'r', 'FontWeight', 'bold');
341
342 %% 11. 输出总结信息
343 fprintf('=== 连续系统控制策略性能分析 ===\n');
344 fprintf('策略1 (LQR最优, K=%.4f): 总代价 = %.4f\n', K_opt, total_cost_opt);
345 fprintf('策略2 (较强反馈, K=%.4f): 总代价 = %.4f\n', K_strong,
    total_cost_strong);
346 fprintf('策略3 (保守控制, K=%.4f): 总代价 = %.4f\n', K_con, total_cost_con);
347 fprintf('开环系统 (无控制): 总代价 = %.4f\n\n', total_cost_open);
348
349 fprintf('闭环系统极点分析:\n');
350 fprintf('策略1 (LQR最优): 闭环极点  $\lambda$  = %.4f, 收敛速率 = %.4f\n', p_opt, -p_opt);
351 fprintf('策略2 (较强反馈): 闭环极点  $\lambda$  = %.4f, 收敛速率 = %.4f\n', p_strong, -
    p_strong);
352 fprintf('策略3 (保守控制): 闭环极点  $\lambda$  = %.4f, 收敛速率 = %.4f\n', p_con, -p_con);
353 fprintf('开环系统: 开环极点  $\lambda$  = %.4f, 收敛速率 = %.4f\n\n', p_open, -p_open);
354
355 fprintf('最优性分析:\n');
356 fprintf('LQR最优策略的总代价最小, 比策略2低 %.2f%%\n', (total_cost_strong-
    total_cost_opt)/total_cost_strong*100);
357 fprintf('LQR最优策略的总代价最小, 比策略3低 %.2f%%\n', (total_cost_con-
    total_cost_opt)/total_cost_con*100);
358 fprintf('LQR最优策略的总代价最小, 比开环系统低 %.2f%%\n\n', (total_cost_open-
    total_cost_opt)/total_cost_open*100);
359
360 fprintf('控制效果分析:\n');
361 fprintf('策略1 (LQR最优): 控制力度适中, 收敛平滑, 代价最小\n');
362 fprintf('策略2 (较强反馈): 控制力度较大, 收敛较快, 但控制能量消耗较多\n');
363 fprintf('策略3 (保守控制): 控制力度很小, 收敛缓慢, 状态误差持续存在时间长\n');
364 fprintf('开环系统: 无控制, 收敛最慢, 代价最大\n');
365

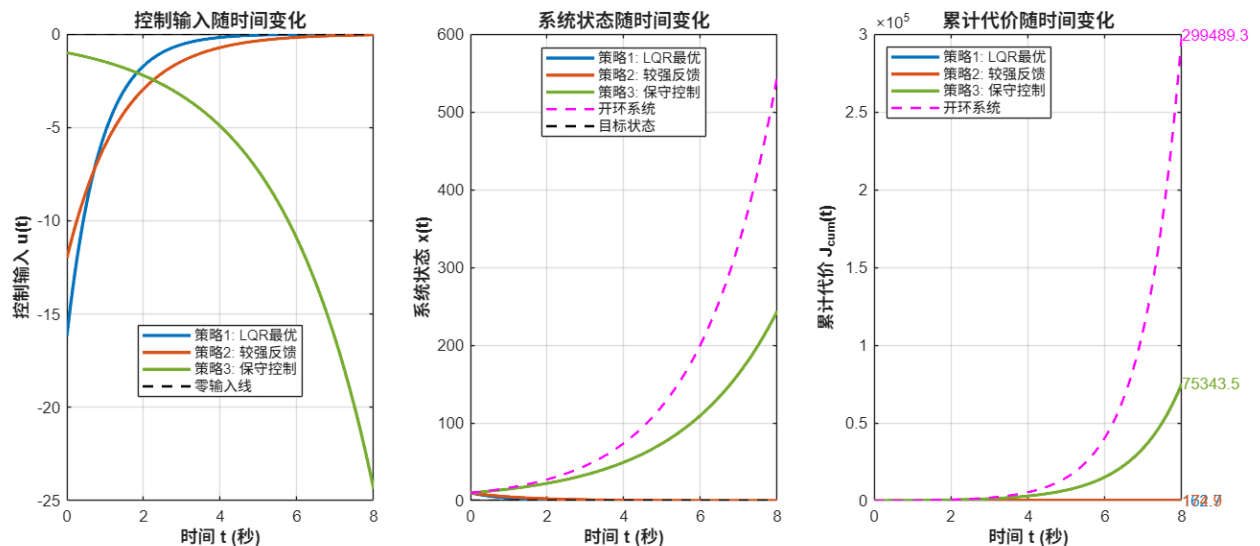
```

```

366 %% 12. 验证解析解与数值解的一致性
367 figure('Color', 'w', 'Position', [400, 200, 800, 400]);
368 subplot(1,2,1);
369 hold on; grid on; box on;
370 plot(t_analytic, x_opt_analytic, 'b-', 'LineWidth', 2, 'DisplayName', '解析解');
371 plot(t, x_opt, 'r--', 'LineWidth', 1.5, 'DisplayName', '数值解');
372 xlabel('时间 t (秒)', 'FontSize', 11, 'FontWeight', 'bold');
373 ylabel('系统状态 x(t)', 'FontSize', 11, 'FontWeight', 'bold');
374 title('LQR最优控制：解析解与数值解对比', 'FontSize', 12, 'FontWeight', 'bold');
375 legend('Location', 'best');
376
377 % 计算相对误差
378 rel_error = abs(x_opt(1:10:end) - interp1(t_analytic, x_opt_analytic,
379     t(1:10:end))) ./ ...
380     abs(interp1(t_analytic, x_opt_analytic, t(1:10:end)));
381
382 subplot(1,2,2);
383 plot(t(1:10:end), rel_error, 'k-', 'LineWidth', 1.5);
384 grid on; box on;
385 xlabel('时间 t (秒)', 'FontSize', 11, 'FontWeight', 'bold');
386 ylabel('相对误差', 'FontSize', 11, 'FontWeight', 'bold');
387 title(sprintf('相对误差 (最大: %.2e)', max_rel_error), 'FontSize', 12,
    'FontWeight', 'bold');

```

连续系统时间域分析：控制输入、系统状态与累计代价



闭环系统极点位置

